

LXNet

Engineering an Honest Evaluation for a Lightweight Chest Radiograph Classifier

A replication study of Humayan et al. (PLOS One, 2026), instrumented to measure and remove near-duplicate leakage in a nine-class chest X-ray corpus, with a full account of the data, training, evaluation and interpretability engineering behind the result.

Author	baveshraam
Repository	https://github.com/baveshraam/LXNet
Subject system	LXNET v1.0.0 — 356,585-parameter CNN, 9-class chest radiograph classifier
Reference work	Humayan et al., <i>A Lightweight CNN for Lung Disease Classification from Chest X-ray with XAI-based Interpretability</i> , PLOS One, 2026
Status	Measured results current as of the <code>runs/grouped</code> and <code>runs/random</code> training arms, seed 42
Scope	Research artefact. Not validated for, and not to be used in, clinical decision-making.

Abstract

The dataset underlying this system contains the same chest radiograph saved repeatedly under different filenames: 6,657 byte-unique image files reduce to 4,635 visually distinct X-rays once near-duplicate copies are collapsed by perceptual hashing — roughly 30% of the corpus is redundant. A model evaluated on a naive random split is scored, in part, on pictures it has already seen during training. Under that protocol, LXNET reaches 95.4% hold-out accuracy; once every re-saved copy of an X-ray is confined to a single split, the same model, the same weights procedure, and the same preprocessing score 92.9%. The 2.5-point gap is not model skill — it is leakage, and it is measured directly in this report rather than assumed.

This document describes the complete system built to make that measurement, and to make every other reported number in the repository trustworthy in the same way: dataset indexing and deduplication, perceptual near-duplicate grouping, group-aware stratified splitting and cross-validation, a cached CLAHE preprocessing pipeline, the LXNET architecture and three transfer-learning baselines (DenseNet201, InceptionV3, ResNet50V2), the training and early-stopping regime, the evaluation and significance-testing methodology, three interpretability methods (Grad-CAM, Score-CAM, LIME), the inference path, and the automated test suite that pins each of these guarantees against regression. Measured results are reported for both the honest (group-aware) and leaky (random) split protocols on identical data, identical preprocessing and identical model code, so the cost of the leak can be read off directly rather than inferred.

The headline engineering trade is explicit: LXNET gives up 5.6 percentage points of accuracy to the strongest baseline (ResNet50V2, 98.5%) in exchange for a $63\times$ improvement in accuracy per parameter (260.5 vs. 4.1 per million parameters). Which side of that trade is correct depends entirely on the deployment target, and this report treats that as an engineering decision rather than a foregone conclusion.

Not a Medical Device

This system is a research artefact evaluated on one public dataset of undocumented clinical provenance. Pneumonia recall is 73.2%: roughly one in four pneumonia films is missed. Nothing in this report should be read as supporting diagnostic, triage, or screening use. See Chapter 13.

Contents

Abstract	i
Glossary of Abbreviations	viii
1 Introduction	1
1.1 Problem Statement	1
1.2 What This System Does	1
1.3 Why This Report Exists	2
1.4 Document Roadmap	2
2 System Architecture	3
2.1 Package Layout	3
2.2 Data Flow	3
2.3 Two Independent Split Protocols, One Codebase	4
2.4 Cross-Cutting Invariant: the Digest-to-Row Map	4
3 Dataset and Data Integrity	5
3.1 Source Corpus	5
3.2 Exact Deduplication	5
3.3 Perceptual Near-Duplicate Grouping	6
3.3.1 Difference Hashing	6
3.3.2 Why the Matching Threshold Is Exact, Not Fuzzy	6
3.3.3 Union-Find Grouping	7
3.4 Group-Aware Splitting	7
3.5 Balancing After Splitting, Not Before	7
3.6 Class Balancing Strategy	9
4 Preprocessing and the Input Pipeline	10
4.1 CLAHE	10
4.2 Caching: Preprocess Once, Not Once per Epoch	10
4.3 The <code>tf.data</code> Pipeline	11
4.3.1 Augmentation Is Conservative by Clinical Reasoning, Not Just Convention	11
5 Model Architecture	12
5.1 LXNET	12
5.1.1 Global Average Pooling Is the Parameter Budget	13
5.2 Transfer-Learning Baselines	13
5.3 The CAM Attachment Point	13

6	Training Methodology	15
6.1	Optimisation Configuration	15
6.2	Hold-Out Training	15
6.3	Cross-Validation and the Early-Stopping Leak	15
6.4	The GPU Guard	16
6.5	Determinism and Seeding	17
7	Evaluation Methodology	18
7.1	Macro-Averaged Metrics	18
7.2	Cross-Fold Summarisation	18
7.3	Paired Significance Testing	18
8	Experimental Results	20
8.1	Headline Finding	20
8.2	Hold-Out Comparison	20
8.3	Does the Small Model Keep Up?	20
8.4	Cross-Validation	22
8.5	Where the Errors Are	23
8.5.1	Confusion Matrices	23
8.6	Training Dynamics	23
8.7	Training Cost	23
8.8	Significance Testing	23
8.9	Dataset After Cleaning	25
9	Explainability	27
9.1	Grad-CAM	27
9.2	Score-CAM	27
9.3	LIME	28
9.4	The Interpretability Panel	28
9.5	Reading the Maps Sceptically	28
10	Inference and Deployment	30
10.1	The Trust Boundary	30
10.2	Preprocessing Parity	30
10.3	The Classifier Interface	31
10.4	Command-Line Interface	31
10.5	Packaging	31
11	Testing and Continuous Integration	32
11.1	Design Philosophy	32
11.2	Continuous Integration	33
12	Engineering Decision Ledger	34
13	Limitations and Ethical Considerations	36
13.1	Intended Use	36
13.1.1	In Scope	36

13.1.2 Explicitly Out of Scope	36
13.2 Numbered Limitations	36
13.3 Training Data Provenance	37
13.4 Ethical Considerations	37
14 Reproducibility	39
14.1 Environment	39
14.2 Reproducing the Reported Numbers	39
14.3 Reproducibility Record	39
14.4 Hardware	40
15 Conclusion and Future Work	41
15.1 Summary	41
15.2 Future Work	41
A Command-Line Interface Reference	42
A.1 <code>lxnet-train</code>	42
A.2 <code>lxnet-predict</code>	42
A.3 <code>lxnet-report</code>	42
A.4 <code>lxnet-explain</code>	42
B Repository Layout	44

List of Figures

2.1	End-to-end data flow. Blue nodes are processing stages; amber nodes are on-disk artifacts; green nodes are terminal outputs. The training flow (top row into <code>train_once</code>) produces a checkpoint that two independent downstream consumers read: the interpretability panel and the inference CLI. Neither consumer re-runs training or re-derives the split.	4
3.1	Both split protocols consume the identical 4,635-group corpus. The only variable is whether a split boundary is allowed to cut through a perceptual group.	8
5.1	LXNET architecture. All convolutions use <code>use_bias=False</code> since batch normalisation makes an explicit bias redundant. Every layer is named so the network graph is addressable by name for interpretability and testing.	12
8.1	Accuracy under each split protocol, per model. Each pair of points is one model measured twice on identical data; the only variable is whether a re-saved copy of a training image was allowed into the test set.	20
8.2	Group-aware test accuracy against parameter count (log scale). LXNET trails the largest backbone by 5.6 percentage points at 1/67 th the parameter count. . .	21
8.3	Honest (group-aware) accuracy by model, ranked.	21
8.4	Per-fold accuracy, 5-fold group-aware cross-validation, LXNET only.	22
8.5	Per-class recall, LXNET, group-aware test set, hardest classes first.	23
8.6	Row-normalised confusion matrices, group-aware test set, all four models. Cell values are percentage of the true class's rows.	24
8.7	Validation accuracy per epoch, group-aware split. Early stopping restores the best-observed epoch per Table 6.1.	26
9.1	The full interpretability panel, one row per class, group-aware test set. Column 1: CLAHE'd input with the true class, correctness, and predicted-class probability. Columns 2–4: Grad-CAM, Score-CAM and LIME for the predicted class. Row 2 is a genuine failure — a Pneumonia film classified as Degenerative Infectious at $p = 0.95$ — shown because a panel of successes only would not demonstrate anything about failure behaviour.	29
11.1	CI pipeline. <code>test</code> and <code>guarantees</code> run in parallel from independent checkouts; both must pass for the workflow to succeed.	33

List of Tables

1	Abbreviations used throughout this report.	viii
2.1	Module responsibilities. Ten modules, no layer of indirection between a CLI flag and the function it configures.	3
3.1	Class distribution, 6,743 images total. Normal outnumbers Chest Changes 2.5:1 — macro-averaged metrics are used throughout (Chapter 7) so this imbalance cannot hide in a headline number.	5
3.2	Size of the largest perceptual-duplicate group as the matching tolerance increases. At threshold 3 the largest group spans 8 of the 9 classes and has a median internal Hamming distance of 14 bits — a transitive-closure artefact of chaining near-matches, not a true duplicate set.	6
3.3	The load-bearing ordering. Swapping steps 4 and 5 reproduces the leak this project exists to measure.	9
4.1	CLAHE configuration, <code>lxnet/preprocess.py</code>	10
4.2	Storing the cache as grayscale <code>uint8</code> rather than float32 RGB is a 12× reduction, entirely because channel replication and float scaling are deferred to the <code>tf.data</code> graph.	11
5.1	All four models, as instantiated by <code>lxnet.models.build_model</code> . Parameter counts are measured via <code>model.count_params()</code> at train time and asserted by <code>tests/test_models.py</code> against a 500,000 budget for LXNET (the reference paper’s claim is ≈ 0.35 M).	13
6.1	Training configuration, identical across all four models and both split protocols. Only the per-model learning rate differs (Table 5.1).	15
8.1	Hold-out comparison across both split protocols. Only LXNET was trained under the random-split arm (Section 8.8); the backbones were run group-aware only, for training-time reasons (Table 8.6).	22
8.2	Every headline metric, group-aware hold-out, all percentages except Macro-F1.	22
8.3	Cross-validation accuracy, both split protocols, identical model and preprocessing. Note the standard deviation: leakage does not only inflate the mean, it more than quadruples the fold-to-fold spread, because how much a random-split fold scores depends on how many of its images happen to have a twin sitting in that fold’s training portion.	23
8.4	Per-class detail, LXNET, group-aware test set.	25
8.5	Recall by class, every model, group-aware test set (%). The weakest cells — Pneumonia and Normal — are the two classes a screening tool exists to separate, and every model gives up most of its accuracy there.	25

8.6	Wall-clock training cost, single GPU, group-aware arm. The last column is deliberately crude (accuracy points per million parameters) but is the reference paper’s argument reduced to one number: LXNET extracts far more accuracy per parameter than any backbone tested, while still finishing below all of them in absolute terms. Note that LXNET trains <i>longer</i> in wall-clock time than two of the three frozen-backbone baselines despite having 50–70× fewer parameters — it is training every weight from scratch, while the baselines are only fitting a small head on already-frozen features.	26
11.1	Test files mapped to the specific regression each one exists to catch, not just the module each one happens to cover.	32
11.2	CI jobs, <code>.github/workflows/ci.yml</code> . Splitting <code>guarantees</code> out of <code>test</code> is a visibility decision, not a technical necessity — both jobs run the same suite on the same commit; the split exists so a reviewer scanning a checks list sees the evaluation-integrity guarantees fail by name.	33
12.1	Engineering decisions with an alternative that was measured or seriously considered and rejected, not merely available in principle.	35
14.1	Pinned dependency versions, <code>pyproject.toml</code>	39
14.2	Reproducibility record. Both arms consume the same cached CLAHE’d tensor (Section 4.2), so preprocessing is bit-for-bit identical between them — the split function is the only variable that differs.	40
A.1	<code>lxnet-train</code> / <code>python -m lxnet.train</code> flags.	42
A.2	<code>lxnet-predict</code> flags.	43
A.3	<code>lxnet-report</code> flags.	43
A.4	<code>lxnet-explain</code> flags.	43

Glossary of Abbreviations

Term	Meaning
CAM	Class Activation Map
CLAHE	Contrast Limited Adaptive Histogram Equalisation
CV	Cross-Validation
dhash	Difference hash (perceptual image fingerprint)
F1 (macro)	Harmonic mean of precision and recall, unweighted across classes
GAP	Global Average Pooling
Grad-CAM	Gradient-weighted Class Activation Mapping
LIME	Local Interpretable Model-agnostic Explanations
LOC	Lines of code
pp	Percentage points
ReLU	Rectified Linear Unit
Score-CAM	Score-weighted Class Activation Mapping
TF	TensorFlow
XAI	Explainable Artificial Intelligence

Table 1: Abbreviations used throughout this report.

CHAPTER 1

Introduction

1.1 Problem Statement

Humayan et al. [1] propose a 0.35-million-parameter convolutional network for classifying chest radiographs into nine disease categories, arguing that a model two orders of magnitude smaller than a standard ImageNet backbone can match its accuracy. That is a strong, falsifiable claim, and it depends entirely on the evaluation being clean: if the test set contains images the model has already seen in some form during training, the accuracy gap between a large and small model collapses for a reason that has nothing to do with either architecture.

The dataset used in the replication — *X-Ray Lung Diseases Images (9 classes)* [7] — turns out to have exactly that problem. Chest radiographs are collected, re-exported, and re-uploaded across medical imaging repositories, and this corpus is no exception: the same film appears multiple times under different filenames, at different JPEG quality, in different color modes. Chapter 3 quantifies this precisely. The consequence for evaluation is direct — a conventional random train/test split does not guarantee that a test image is unseen, only that its *filename* was not used for training.

1.2 What This System Does

LXNET is both a re-implementation of the architecture proposed in [1] and an evaluation harness built specifically to answer one question honestly: *once every near-duplicate copy of an X-ray is kept on one side of the split, how much of the reported accuracy survives?* The system:

1. indexes, hashes, and deduplicates the raw dataset (Chapter 3);
2. builds a perceptual near-duplicate graph and derives a *group-aware* stratified split and *k*-fold cross-validation, so no evaluation image has a training-set twin (Chapter 3);
3. preprocesses every image once with CLAHE and caches the result, rather than repeating an expensive CPU-bound transform every epoch (Chapter 4);
4. trains LXNET and three transfer-learning baselines — DenseNet201, InceptionV3, ResNet50V2 — under an identical protocol (Chapters 2 and 6);
5. evaluates every model with macro-averaged metrics and a paired significance test across cross-validation folds (Chapter 7);
6. explains individual predictions with Grad-CAM, Score-CAM and LIME (Chapter 9);
7. exposes a validated, file-based inference path with an explicit trust boundary (Chapter 10); and
8. pins every one of the above guarantees with an automated test suite that runs in continuous integration (Chapter 11).

1.3 Why This Report Exists

The repository's `README.md` and `docs/RESULTS.md` answer *what the numbers are*. This report answers *why the system is built the way it is* — the internal data structures, the ordering constraints between deduplication, splitting and balancing, the reasoning behind each hyperparameter and architectural choice, and the specific regression each test in the suite exists to catch. It is written for an engineer who needs to modify, extend, or audit the system, not only read its headline numbers.

1.4 Document Roadmap

Chapter 2 gives the system-level view: module responsibilities, their dependencies, and the two data flows (training and inference) that thread through them. Chapters 3 and 4 cover the data layer in depth — indexing, deduplication, perceptual grouping, splitting, and the cached preprocessing pipeline. Chapters 5 to 7 cover the model, the training loop, and the metrics. Chapter 8 reports measured outcomes. Chapters 9 and 10 cover interpretability and deployment. Chapter 11 maps the test suite to the guarantees it protects. Chapter 12 is a standalone ledger of the non-obvious engineering trade-offs made throughout, with rationale. Chapter 13 states, without hedging, what this system does not know how to do. Chapter 14 gives the exact commands and seeds needed to regenerate every number in this report.

CHAPTER 2

System Architecture

2.1 Package Layout

The system is a single installable Python package, `lxnet`, exposing four console entry points (`lxnet-train`, `lxnet-predict`, `lxnet-report`, `lxnet-explain`) over ten modules. There is no service layer, no configuration-file system, and no plugin architecture — every module is imported directly by the module that needs it, and every tunable parameter is a CLI flag or a function default. For a system with one dataset, one architecture family, and one deployment target (a local checkpoint file), this is a deliberate ceiling: an abstraction layer for multiple datasets or model registries would be undirected complexity with no second consumer to justify it.

Module	Responsibility
<code>data.py</code>	Dataset indexing, SHA-256 deduplication, perceptual (dhash) near-duplicate grouping, group-aware stratified splitting and k -fold cross-validation, class balancing.
<code>preprocess.py</code>	CLAHE contrast equalisation and single-image loading for inference.
<code>pipeline.py</code>	Bulk CLAHE preprocessing into a cached uint8 tensor; the <code>tf.data</code> pipeline that serves batches to training and evaluation.
<code>models.py</code>	The LXNET architecture and the three transfer-learning baselines, behind one dispatch function.
<code>train.py</code>	CLI orchestration: hold-out training, k -fold cross-validation, the GPU guard, and <code>results.json</code> generation.
<code>evaluate.py</code>	Metric computation (macro precision/recall/F1, confusion matrix), per-fold summarisation, Wilcoxon signed-rank testing.
<code>xai.py</code>	Grad-CAM, Score-CAM and LIME implementations, plus heatmap overlay compositing.
<code>explain.py</code>	CLI driver that renders the per-class interpretability panel from a trained checkpoint.
<code>predict.py</code>	The inference trust boundary: input validation, the <code>Classifier</code> class, and the <code>lxnet-predict</code> CLI.
<code>report.py</code>	Turns <code>results.json</code> into every figure in <code>docs/</code> (light and dark themed) and the generated <code>docs/RESULTS.md</code> .

Table 2.1: Module responsibilities. Ten modules, no layer of indirection between a CLI flag and the function it configures.

2.2 Data Flow

Figure 2.1 traces both flows through the system: the training flow that produces a scored checkpoint, and the two consumers of that checkpoint — the interpretability panel and the inference CLI. Every arrow in the diagram is a real function call or file artifact in the codebase; none is aspirational.

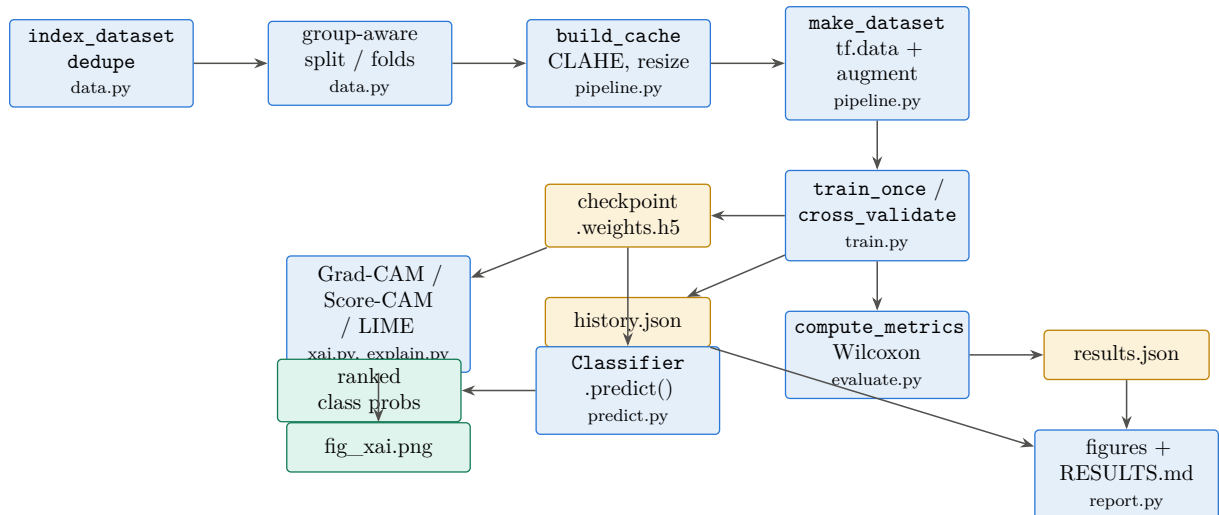


Figure 2.1: End-to-end data flow. Blue nodes are processing stages; amber nodes are on-disk artifacts; green nodes are terminal outputs. The training flow (top row into `train_once`) produces a checkpoint that two independent downstream consumers read: the interpretability panel and the inference CLI. Neither consumer re-runs training or re-derives the split.

2.3 Two Independent Split Protocols, One Codebase

A structural decision worth calling out on its own: `train.py` accepts `--split-mode {grouped,random}` and routes to either `group_aware_split/group_aware_folds` or the naive `stratified_split` in `data.py`. The random path is not legacy code kept for compatibility — it is retained *specifically* so the two protocols can be run on identical images, identical preprocessing, and identical model code, differing only in which split function partitions the data. Chapter 8 depends on this: the leakage measurement is only meaningful because both arms share every other variable.

2.4 Cross-Cutting Invariant: the Digest-to-Row Map

Every module downstream of `pipeline.build_cache` refers to images by integer row index into a single cached tensor (`runs/<name>/preprocessed.npz`), not by file path. `train.py` maintains a `digest_to_row` dictionary (SHA-256 digest $\rightarrow$ cache row) to translate between the `Sample` objects that `data.py` works with — which carry semantic identity (path, label, digest) — and the row indices that `pipeline.py` and `models.py` consume. This indirection exists because splitting, grouping and folding are naturally expressed over `Sample` objects, while the actual tensors fed to Keras are naturally expressed as flat arrays. Any code that reintroduces a positional assumption between the two (for instance, assuming cache row i corresponds to `samples[i]` after a re-order) reintroduces the exact image/label misalignment bug that `tests/test_pipeline.py` exists to catch (Chapter 11).

CHAPTER 3

Dataset and Data Integrity

3.1 Source Corpus

The corpus is *X-Ray Lung Diseases Images (9 classes)* [7]: 6,743 image files under nine class directories, named with a two-digit numeric prefix (00–08) followed by a Portuguese class name. `data.py` sorts class directories by name and assigns integer labels 0–8 by that sort order — so the numeric prefix, not the display label, is load-bearing; renaming a class directory without preserving its prefix silently relabels every image in it.

Label	Class	Images	Label	Class	Images
0	Normal	1340	5	Degenerative Infectious	594
1	Pneumonia	1060	6	Encapsulated Lesions	658
2	Higher Density	678	7	Mediastinal Changes	596
3	Lower Density	629	8	Chest Changes	544
4	Obstructive Pulmonary	644			

Table 3.1: Class distribution, 6,743 images total. Normal outnumbers Chest Changes 2.5:1 — macro-averaged metrics are used throughout (Chapter 7) so this imbalance cannot hide in a headline number.

An on-load audit (exercised by `tests/test_data.py`) finds zero corrupt files, 40 distinct image resolutions (6,263 of the 6,743 files are 450×450), and a mix of grayscale, RGB and RGBA colour modes — consistent with images aggregated from more than one export pipeline rather than one consistent acquisition system.

3.2 Exact Deduplication

Every file is hashed with SHA-256 on load (`data._digest`). Grouping by digest finds 74 groups of byte-identical files, 84 excess copies in total, and — more seriously — one image whose byte-identical copies are filed under two contradictory classes (*Pneumonia* and *Lower Density*). `data.dedupe` drops *both* copies of a cross-class conflict rather than picking one: keeping either would train the model on a coin flip for that exact image, and there is no principled way to decide which label is correct from the pixels alone.

```
1 for group in by_digest.values():
2     labels = {s.label for s in group}
3     if len(labels) > 1:
4         report.cross_class_conflicts += 1
5         log.warning(
6             "dropping %d copies of an image labelled as %s",
7             len(group), sorted(s.class_name for s in group),
8         )
```

```

9         continue
10     report.exact_duplicates += len(group) - 1
11     kept.append(group[0])

```

Listing 3.1: Cross-class conflicts are dropped entirely, not resolved by a heuristic (lxnet/data.py).

Exact hashing only catches files that are bit-for-bit identical. It misses the more common case on this dataset: the same X-ray, re-exported at a different JPEG quality or resized, which flips most of the file’s bytes while leaving the image visually unchanged to a clinician — and to a convolutional network.

3.3 Perceptual Near-Duplicate Grouping

3.3.1 Difference Hashing

Visual identity is approximated with a *difference hash* (dhash): each image is converted to grayscale, downscaled to a 9×8 thumbnail, and encoded as a 64-bit signature where bit (i, j) is 1 if pixel $(i, j+1)$ is brighter than pixel (i, j) . Downscaling to 9×8 deliberately discards resolution, compression artefacts and sensor noise, so a re-saved or resized copy of the same film lands on the same, or a very close, digest.

```

1 def _dhash(path_str: str, hash_size: int = 8) -> str:
2     with Image.open(path_str) as im:
3         thumb = im.convert("L").resize(
4             (hash_size + 1, hash_size), Image.LANCZOS
5         )
6     a = np.asarray(thumb, dtype=np.int16)
7     return np.packbits((a[:, 1:] > a[:, :-1]).flatten()).tobytes().hex()

```

Listing 3.2: The difference hash (lxnet/data.py).

3.3.2 Why the Matching Threshold Is Exact, Not Fuzzy

The natural extension — treat two digests within a small Hamming distance as the same image — was measured and rejected. Chest radiographs are near-identical *by construction*: same anatomy, same framing convention, low dynamic range. Table 3.2 shows what happens to the largest connected component as the tolerance is relaxed.

Hamming threshold (bits)	0	1	2	3
Largest resulting group (images)	21	143	374	1,680

Table 3.2: Size of the largest perceptual-duplicate group as the matching tolerance increases. At threshold 3 the largest group spans 8 of the 9 classes and has a median internal Hamming distance of 14 bits — a transitive-closure artefact of chaining near-matches, not a true duplicate set.

At threshold 0 (exact digest equality), sampled matching pairs have a median pixel correlation of 0.9913, against 0.4263 for random image pairs — strong evidence that an exact-digest match really is the same film re-saved, and that the grouping does not need, and should not use, any tolerance at all. `GROUP_MAX_DISTANCE = 0` in `data.py` encodes this conclusion as a constant, with the measurement recorded alongside it in a comment rather than left implicit.

3.3.3 Union-Find Grouping

Even at threshold 0, some images occupy more than one digest bucket after a resize (a genuinely-tiny compression difference can flip a hash bit). `group_samples` therefore performs a full pairwise Hamming comparison between all distinct digests, in $256 \times N$ blocks to bound memory, and unions any pair within `max_distance` using a union-find structure:

```

1 chunk = 256
2 for start in range(0, len(digests), chunk):
3     block = bits[start : start + chunk]
4     dist = _POPCOUNT[block[:, None, :] ^ bits[None, :, :]].sum(axis=2)
5     for local_i, row in enumerate(dist):
6         i = start + local_i
7         for j in np.nonzero(row <= max_distance)[0]:
8             if j > i:
9                 union(i, int(j))

```

Listing 3.3: Blocked pairwise Hamming distance plus union-find (`lxnet/data.py`).

With `max_distance = 0` this loop is doing more work than strictly necessary — an exact-match grouping could be a single dictionary keyed by digest — but the general form is what makes the threshold in Table 3.2 a one-line experiment rather than a rewrite, and the cost is bounded ($O(N^2/256)$ comparisons over $N \approx 6,600$ digests, each an 8-byte XOR-and-popcount) and pays for itself once at indexing time, not once per training run.

3.4 Group-Aware Splitting

Figure 3.1 lays out the full pipeline from raw files to a scored split, and the two protocols this system supports at the last step.

`stratified_split` splits individual *images*: it shuffles each class’s samples and slices by fraction, which guarantees per-class proportions but says nothing about which images end up on which side of the cut. `group_aware_split` instead treats each perceptual group as an indivisible unit, assigns it the label its majority member carries, and deals whole groups to train/val/test with a greedy quota rule: at each step, the next group goes to whichever split is furthest below its target share of the running total.

```

1 for idx in order:
2     members = entries[idx]
3     total = sum(counts.values()) + len(members)
4     choice = max(targets, key=lambda k: targets[k] * total - counts[k])
5     parts[choice].extend(members)
6     counts[choice] += len(members)

```

Listing 3.4: Greedy quota assignment keeps every split near its target proportion while never splitting a group (`lxnet/data.py:group_aware_split`).

k -fold cross-validation uses the same grouping through `sklearn.model_selection.StratifiedGroupKFold` [5], passing each sample’s group id alongside its label so every fold boundary is also group-clean.

3.5 Balancing After Splitting, Not Before

The reference paper balances classes before splitting. On a dataset with 30% redundancy, that ordering places duplicated copies of a minority-class image on both sides of the eventual split — oversampling manufactures the exact leakage this system exists to remove, concentrated on

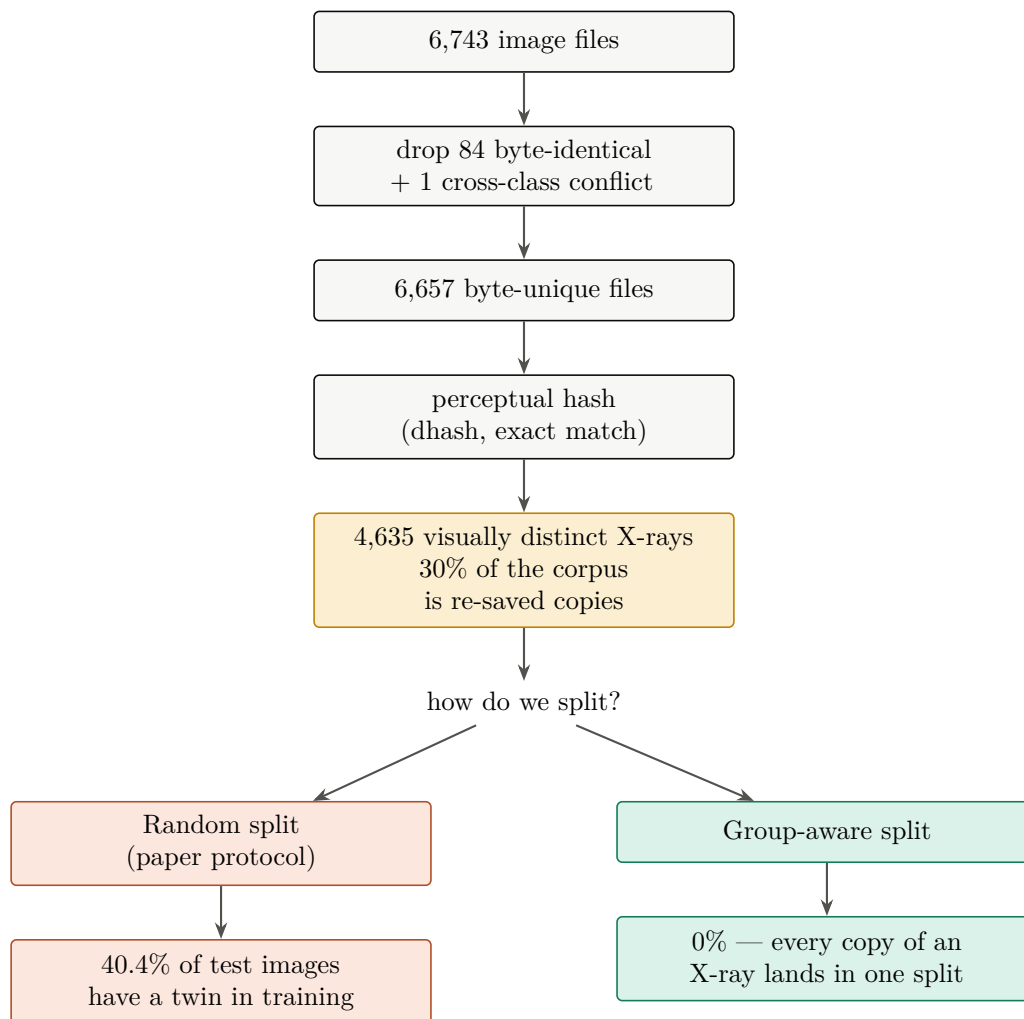


Figure 3.1: Both split protocols consume the identical 4,635-group corpus. The only variable is whether a split boundary is allowed to cut through a perceptual group.

the classes an evaluator would trust least. `train.py` enforces the opposite order unconditionally: split first, then oversample the training portion only. Table 3.3 makes this explicit as an ordering constraint rather than a convention.

Step	Operation	Why this position in the sequence
1	Index + SHA-256 hash	Establishes exact-duplicate identity before anything touches labels.
2	Deduplicate	Removes byte-identical copies and cross-class conflicts; must precede perceptual grouping so groups are built on the reduced set.
3	Perceptual grouping	Establishes which files are the same X-ray, independent of the eventual split.
4	Group-aware split	Deals whole groups to train/val/test. Must precede balancing.
5	Balance (oversample)	Applied to the <i>training</i> split's rows only.
6	Cache + train	The balanced, split-correct index is the only thing handed to <code>pipeline.build_cache</code> for training.

Table 3.3: The load-bearing ordering. Swapping steps 4 and 5 reproduces the leak this project exists to measure.

Cross-validation applies the same discipline one level deeper: each fold's early-stopping monitor is carved out of that fold's *training* rows before oversampling, so a duplicated minority-class row can never straddle the monitor and the training set within a fold either. This is covered in detail in Section 6.3.

3.6 Class Balancing Strategy

Two balancing mechanisms exist in `data.py`: `oversample_to_balance`, which duplicates minority-class rows uniformly at random up to the majority count, and `class_weights`, which computes inverse-frequency sample weights normalised to a mean of approximately 1. Only oversampling is wired into the training CLI — `train.py` calls `_oversample_indices` on the training split and passes `class_weight=None` to `model.fit`. `class_weights` is correct, tested (`tests/test_data.py`), and unused by default: it remains available as a zero-duplication alternative for a future run that wants to avoid inflating the training set's memory footprint, but changing the default requires a deliberate decision, not a silent switch.

Preprocessing and the Input Pipeline

4.1 CLAHE

The reference paper’s headline preprocessing step is Contrast Limited Adaptive Histogram Equalisation. A chest radiograph carries its diagnostic signal in a narrow band of greys, and global histogram equalisation over- or under-corrects regions with very different local contrast (dense bone versus soft lung parenchyma in the same frame). CLAHE addresses this by equalising within local tiles and clipping the contrast amplification so that flat, low-signal regions are not turned into noise.

`preprocess.apply_clahe` wraps `cv2.createCLAHE` with a clip limit of 2.0 and an 8×8 tile grid (Table 4.1), applied to a single-channel grayscale image. Source files are collapsed to grayscale *before* CLAHE runs: applying CLAHE independently per RGB channel would equalise each channel’s contrast separately and shift the overall colour balance, which is meaningless on a modality that carries no colour information in the first place.

Parameter	Value	Effect
<code>clipLimit</code>	2.0	Ceiling on local contrast amplification; higher values amplify sensor/compression noise in flat regions.
<code>tileGridSize</code>	8×8	Number of local regions the image is independently equalised over.
Input	2-D <code>uint8</code>	CLAHE is defined over a single intensity channel.
Output resolution	224×224	Matches the input contract of all four model architectures (Chapter 5).

Table 4.1: CLAHE configuration, `lxnet/preprocess.py`.

After equalisation, images are resized to 224×224 with `cv2.INTER_AREA` (the appropriate interpolation kernel for downsampling, as most source images exceed the target resolution), scaled to $[0, 1]$, and replicated across three channels. The channel replication is what lets grayscale LXNET and the ImageNet-pretrained RGB baselines consume literally the same tensor — a prerequisite for the parameter-count comparison in Chapter 8 to be a fair one.

4.2 Caching: Preprocess Once, Not Once per Epoch

CLAHE runs on the CPU through OpenCV and is far too slow to repeat on every training epoch. `pipeline.build_cache` therefore decodes, CLAHE’s and resizes every sample exactly once, storing the result as an $(N, 224, 224)$ `uint8` grayscale array at `runs/<name>/preprocessed.npz`. Table 4.2 quantifies why the array is grayscale `uint8` and not float32 RGB: channel replication and the $[0, 1]$ cast happen inside the `tf.data` graph, on the already-cached tensor, not before it is written to disk.

One implementation detail is load-bearing on Windows specifically: `cv2.imread` cannot open a path containing non-ASCII characters, and the source dataset’s class directories are named in Portuguese. `build_cache` reads the file’s bytes directly with `np.fromfile` and decodes them with

Representation	Bytes per image	Full corpus (6.7k images)
Grayscale <code>uint8</code> , cached	$224 \times 224 \times 1 = 50,176$	≈ 338 MB
Float32 RGB, uncached	$224 \times 224 \times 3 \times 4 = 602,112$	≈ 4 GB

Table 4.2: Storing the cache as grayscale `uint8` rather than float32 RGB is a $12\times$ reduction, entirely because channel replication and float scaling are deferred to the `tf.data` graph.

`cv2.imdecode`, sidestepping the OS path-handling layer entirely:

```

1 buffer = np.fromfile(sample.path, dtype=np.uint8)
2 gray = cv2.imdecode(buffer, cv2.IMREAD_GRAYSCALE)
3 if gray is None:
4     raise ValueError(f"could not decode image: {sample.path}")

```

Listing 4.1: Reading by buffer, not by path, to survive non-ASCII directory names (`lxnet/pipeline.py`).

If the cache file already exists, `build_cache` loads it directly and skips reprocessing; the file must be deleted manually to force a rebuild after a preprocessing change (Chapter 12 discusses the trade-off this implies).

4.3 The `tf.data` Pipeline

`pipeline.make_dataset` wraps the cached array in a `tf.data` pipeline that performs, in order: cast to `float32` and scale to $[0, 1]$, expand to a single channel dimension and replicate to three, optional augmentation, batching, and prefetch with `AUTOTUNE`. Training datasets are shuffled with a buffer of $\min(N, 4096)$ elements, reshuffled every epoch; evaluation datasets are never shuffled, because downstream code matches predictions to labels by array position, not by an explicit key.

4.3.1 Augmentation Is Conservative by Clinical Reasoning, Not Just Convention

```

1 def _augment(x: tf.Tensor, seed: int) -> tf.Tensor:
2     x = tf.image.random_flip_left_right(x)
3     x = tf.image.random_brightness(x, max_delta=0.10)
4     x = tf.image.random_contrast(x, lower=0.90, upper=1.10)
5     return tf.clip_by_value(x, 0.0, 1.0)

```

Listing 4.2: The full augmentation policy (`lxnet/pipeline.py`).

Horizontal flip, and mild brightness/contrast jitter ($\pm 10\%$ and $0.90\text{--}1.10\times$ respectively) — and nothing else. Vertical flip and large-angle rotation are deliberately excluded: an upside-down chest radiograph is not a plausible clinical acquisition, and left-right mirroring is not always augmentation-neutral either, since dextrocardia and situs inversus are themselves diagnostic findings that a naively mirrored image could misrepresent. Augmentation is applied only when `training=True`, and only inside the `tf.data` map stage — it never touches the cached array on disk, so validation and test batches drawn from the same cache are guaranteed augmentation-free.

CHAPTER 5

Model Architecture

5.1 LXNet

LXNET is three VGG-style convolutional blocks (32, 64, 128 filters) each followed by batch normalisation, ReLU and max-pooling, then global average pooling and a two-layer classification head. Figure 5.1 shows the full stack with spatial dimensions at each stage; ?? gives the exact, test-verified parameter count.

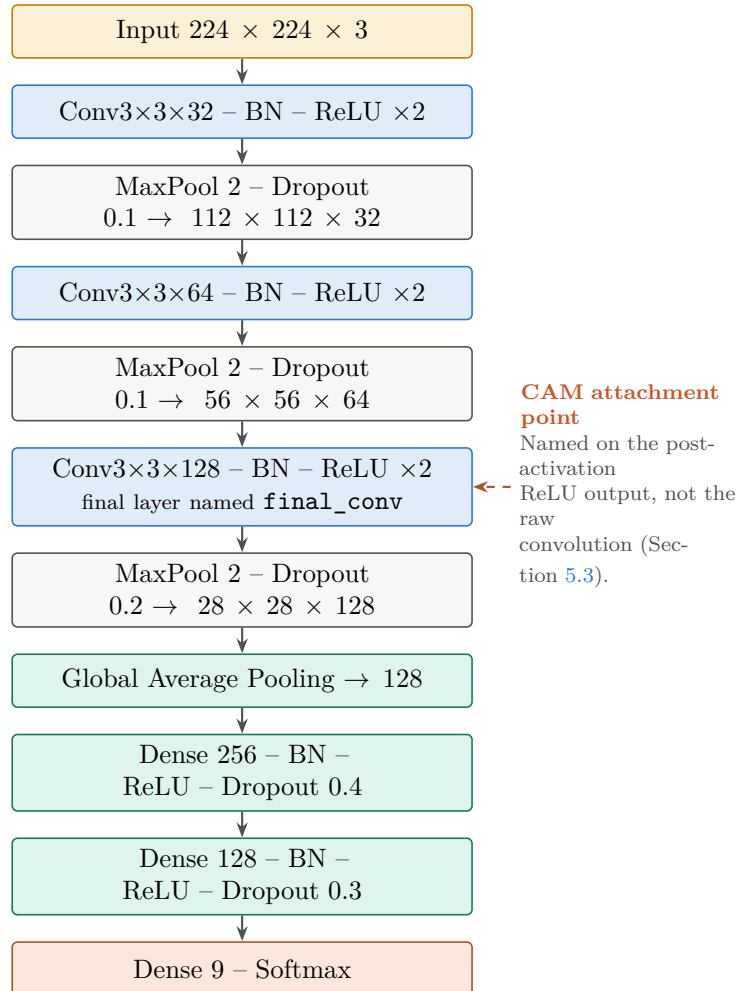


Figure 5.1: LXNET architecture. All convolutions use `use_bias=False` since batch normalisation makes an explicit bias redundant. Every layer is named so the network graph is addressable by name for interpretability and testing.

5.1.1 Global Average Pooling Is the Parameter Budget

The single architectural decision responsible for LXNET’s parameter count is using global average pooling instead of flattening before the classifier head. Flattening the final $28 \times 28 \times 128$ feature map and feeding it to a 256-unit dense layer would cost $28 \times 28 \times 128 \times 256 = 25,690,112$ weights on that one layer alone — more than 70 times the entire network’s actual parameter count. Global average pooling collapses each of the 128 channels to a single scalar first, so the same dense layer costs $128 \times 256 = 32,768$ weights instead.

5.2 Transfer-Learning Baselines

Three ImageNet-pretrained backbones are wired behind an identical classifier head (global average pooling, Dense-256-ReLU, Dropout 0.4, Dense-9-softmax), frozen by default (`trainable_backbone=False`). The backbone is frozen because the comparison of interest is *LXNET trained from scratch* against the standard “pretrained backbone plus a new head” recipe most practitioners would actually reach for — and a fair comparison needs an identical head shape across all four models, which `build_baseline` enforces structurally rather than by convention.

Model	Parameters	Pretrained	Backbone trainable	Learning rate
LXNET	356,585	No (from scratch)	—	1×10^{-3}
DenseNet201	18,816,073	ImageNet	Frozen	1×10^{-4}
InceptionV3	22,329,641	ImageNet	Frozen	1×10^{-4}
ResNet50V2	24,091,657	ImageNet	Frozen	1×10^{-4}

Table 5.1: All four models, as instantiated by `lxnet.models.build_model`. Parameter counts are measured via `model.count_params()` at train time and asserted by `tests/test_models.py` against a 500,000 budget for LXNET (the reference paper’s claim is ≈ 0.35 M).

LXNET trains at $10\times$ the learning rate of the baselines because it trains every weight from a random initialisation, while the baselines are only fitting a small randomly-initialised head on top of frozen, already well-conditioned ImageNet features — a higher rate on the frozen-backbone models would risk destabilising the head before it has had a chance to converge against fixed features.

5.3 The CAM Attachment Point

The final convolutional block’s activation layer is explicitly named `final_conv`, and this is not cosmetic — it is the layer every interpretability method in Chapter 9 attaches to, and its position in the Conv $\rightarrow$ BatchNorm $\rightarrow$ Activation sequence matters:

- the **raw convolution output** is pre-BatchNorm, so its per-channel scale is arbitrary and not comparable across channels;
- it is also **signed**, so a channel’s negative activations survive into a weighted sum in a way Grad-CAM’s formulation does not account for;
- the **post-activation** (post-ReLU) output is what Grad-CAM [2] is defined over: non-negative, batch-normalised, and directly comparable across channels.

`xai._feature_layer` raises a `ValueError` if the requested layer name is not found in the model, rather than silently falling back to a default — an earlier version of this code returned

the default unchecked, which meant a model missing the expected attachment point failed deep inside `model.get_layer` with no indication of why.

CHAPTER 6

Training Methodology

6.1 Optimisation Configuration

Setting	Value
Optimiser	Adam
Loss	Sparse categorical cross-entropy
Batch size	32
Max epochs	40
Early stopping	monitor <code>val_loss</code> , patience 8, <code>restore_best_weights=True</code>
LR schedule	<code>ReduceLROnPlateau</code> , monitor <code>val_loss</code> , factor 0.5, patience 2, min LR 1×10^{-6}
Checkpointing	best <code>val_loss</code> only, weights only
Seed	42 (<code>tf.keras.utils.set_random_seed</code>)

Table 6.1: Training configuration, identical across all four models and both split protocols. Only the per-model learning rate differs (Table 5.1).

6.2 Hold-Out Training

`train.train_once` builds fresh `tf.data` pipelines for the train, validation and test index arrays, fits with the callback stack in Table 6.1, and scores the fitted model on the held-out test indices exactly once. `tf.keras.backend.clear_session()` is called before every model build so that successive runs in the same process (four models per hold-out pass, then up to five more per cross-validation fold) do not accumulate graph state across models with different architectures.

6.3 Cross-Validation and the Early-Stopping Leak

k -fold cross-validation raises a subtler correctness problem than the train/test split does. Early stopping with `restore_best_weights` does not simply train for a fixed number of epochs — it trains for up to 40 epochs and *keeps whichever epoch’s weights scored best on the validation set it is watching*. If that validation set is the same fold being reported as the cross-validation score, the reported number is not “the model’s accuracy” but “the maximum of the model’s accuracy over up to 40 draws” — a systematic optimistic bias that grows with the patience budget.

`train._carve_validation` avoids this by splitting each fold’s *training* rows a second time, stratified by class, into an inner training set and a small early-stopping monitor (15% by default, with at least one example per class held back where the class is large enough). The held-out fold that is actually scored is never touched until the model has already stopped training on it.

```
1 def _carve_validation(  
2     idx: np.ndarray, labels: np.ndarray, seed: int, fraction: float =  
    0.15
```

```

3 ) -> tuple[np.ndarray, np.ndarray]:
4     """Split fold-training rows into (train, early-stopping monitor),
        stratified.
5
6     ponytail: carves by row, not by perceptual group, so a near-duplicate
        may
7     straddle the inner boundary. That only blunts the stopping-epoch
8     choice --
9     the reported score comes from the outer fold, which stays group-clean
10    .
    Carve by group if the stopping epoch ever looks systematically late.
    """

```

Listing 6.1: The inner train/monitor split that keeps early-stopping honest (lxnet/train.py).

Documented Engineering Debt

`_carve_validation` splits by individual cache row, not by perceptual group, so a near-duplicate pair can straddle the inner train/monitor boundary within a fold. This is deliberately left as-is: the boundary it affects only influences *which epoch* early stopping selects, not the score that gets reported — the outer, scored fold remains perceptual-group-clean regardless. The upgrade path, if the stopping epoch is ever observed to be systematically too late (a symptom of the monitor being easier than it should be), is to carve by group using the same machinery as Section 3.4.

Oversampling is applied *after* this inner carve, and only to the inner training rows — the same ordering constraint from Section 3.5, applied one level deeper so that a duplicated minority-class row cannot appear in both the inner training set and the monitor set within a single fold.

6.4 The GPU Guard

TensorFlow 2.10 on Windows locates the CUDA runtime through the active Conda environment’s `Library/bin` directory on `PATH`, which `conda activate` sets up. Invoking the environment’s `python.exe` by an absolute path (for example, from a scheduled task or an IDE run configuration that does not source the activation script) silently falls back to CPU — training still runs, still produces a model, and turns a 40-minute job into an overnight one with no error message anywhere in the log.

`train.main` refuses to start training on CPU unless `--allow-cpu` is passed explicitly:

```

1 if not gpus and not args.allow_cpu:
2     raise SystemExit(
3         "no GPU visible to TensorFlow -- refusing to train on CPU.\n"
4         "Activate the env (`conda activate lxnet`) so CUDA is on PATH, "
5         "or pass --allow-cpu."
6     )

```

Listing 6.2: Fail fast rather than discover a CPU fallback the next morning (lxnet/train.py).

This is a case where the lazy failure mode (silently doing the slow thing) is strictly worse than the loud one: a training run that aborts in the first second is a two-second fix, while one that silently spends nine hours on CPU is discovered only when someone checks on it.

6.5 Determinism and Seeding

A single seed (default 42) drives every source of randomness in a run: `tf.keras.utils.set_random_seed` for the model’s weight initialisation and TensorFlow-internal shuffling, and `numpy.random.default_rng(seed)` for every split, grouping-tie-break and oversampling decision in `data.py` and `train.py`. Cross-validation folds derive per-fold seeds as `seed + fold` for the inner carve and oversampling steps, so each fold’s inner split is reproducible but not identical across folds. Full bit-for-bit reproducibility across machines is not claimed — cuDNN convolution algorithms are not guaranteed deterministic across GPU models and driver versions — but every source of randomness under this codebase’s control is seeded and logged.

CHAPTER 7

Evaluation Methodology

7.1 Macro-Averaged Metrics

Every reported precision, recall and F1 score in this system is macro-averaged: computed independently per class, then averaged with equal weight regardless of class support. Table 3.1 showed why this is not a stylistic choice — Normal (1,340 images) outnumbered Chest Changes (544 images) by 2.5:1, and a micro-averaged (support-weighted) metric would let a model that neglects the smallest classes still post a respectable headline number, because those classes contribute little to the weighted total. Macro averaging forces every class's performance to matter equally to the reported score, which is the property an evaluator actually wants when the classes are not clinically interchangeable.

`evaluate.compute_metrics` wraps `sklearn.metrics.precision_recall_fscore_support` with `average="macro"` and `zero_division=0` (so a class with zero predicted or zero true instances contributes 0 rather than raising or silently excluding itself), and always returns the full confusion matrix alongside the scalar metrics — every per-class table and confusion-matrix figure in Chapter 8 is derived from this one matrix, not recomputed independently.

7.2 Cross-Fold Summarisation

`evaluate.summarise_folds` computes the mean and population standard deviation of every numeric metric across a list of per-fold result dictionaries, skipping non-numeric fields automatically (so it can be handed a fold's full metrics dictionary, including the confusion matrix, without special-casing which keys are summarisable).

7.3 Paired Significance Testing

Comparing two models' cross-validation scores is a paired-samples problem: both models are scored on the *same* folds, so the correct test compares per-fold differences rather than treating the two score distributions as independent samples. With only five folds, assuming those differences are normally distributed is not defensible, so `evaluate.wilcoxon_compare` uses the Wilcoxon signed-rank test [6] — non-parametric, and appropriate for small paired samples.

```
1 def wilcoxon_compare(a, b, alpha: float = ALPHA) -> dict:
2     differences = a - b
3     if np.allclose(differences, 0):
4         # scipy raises on an all-zero difference vector;
5         # the answer is "no effect".
6         return {"statistic": 0.0, "p_value": 1.0,
7                 "significant": False, "median_difference": 0.0}
8     statistic, p_value = wilcoxon(a, b)
9     return {
```

```
10     "statistic": float(statistic), "p_value": float(p_value),  
11     "significant": bool(p_value < alpha),  
12     "median_difference": float(np.median(differences)),  
13 }
```

Listing 7.1: Paired, non-parametric comparison across cross-validation folds (`lxnet/evaluate.py`).

The all-zero-difference guard is not a hypothetical edge case: SciPy’s Wilcoxon implementation raises on a rank-degenerate input, and two identical score vectors is exactly the input a unit test would use to check “no difference reports as no difference” — the function needs to handle that input correctly, not just the well-behaved ones. As reported in Section 8.8, only LXNET was cross-validated in the measured run, so this comparison currently has one populated arm.

CHAPTER 8

Experimental Results

All results in this chapter are measured, not projected: both the group-aware and random arms were trained end-to-end with the configuration in Table 6.1, and every figure below is generated directly from the resulting `results.json` by `lxnet.report` — none is hand-drawn (Section 14.3).

8.1 Headline Finding

Under a random split, 40.4% of test images have a near-duplicate in the training set; under the group-aware split, none do. LXNET scores 92.9% on the group-aware test set against 95.4% on the random split — a 2.5 percentage-point gain that is attributable entirely to leakage, since both numbers come from the same model, the same weights, and the same preprocessing. The strongest model overall, on the honest split, is ResNet50V2 at 98.5%.

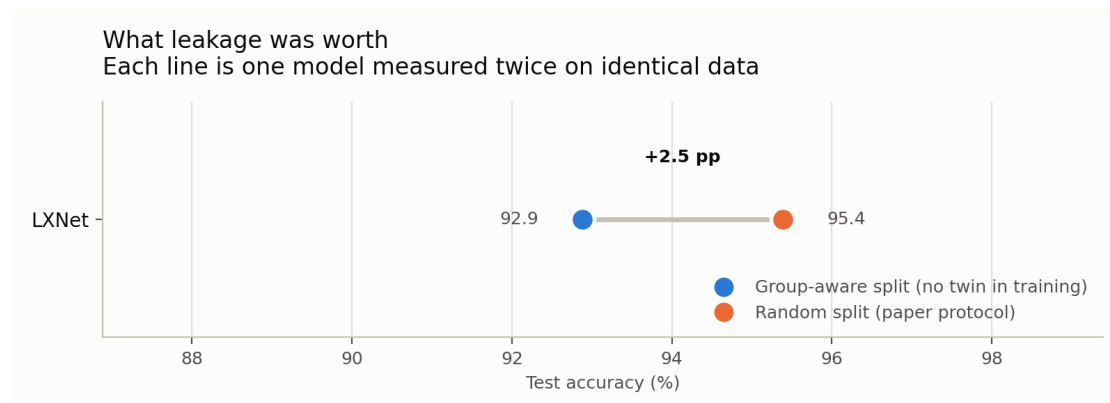


Figure 8.1: Accuracy under each split protocol, per model. Each pair of points is one model measured twice on identical data; the only variable is whether a re-saved copy of a training image was allowed into the test set.

8.2 Hold-Out Comparison

8.3 Does the Small Model Keep Up?

The reference paper’s central claim is that ≈ 0.35 M parameters can match backbones $50\text{--}70\times$ larger. That claim is only testable on a split where the models can actually be told apart — under enough leakage, every model’s accuracy compresses toward the ceiling the dataset allows, and the parameter-efficiency argument becomes unfalsifiable.

The trade is explicit: LXNET gives up 5.6 percentage points of accuracy to ResNet50V2 and returns 260.5 accuracy points per million parameters against ResNet50V2’s 4.1 — a $63\times$

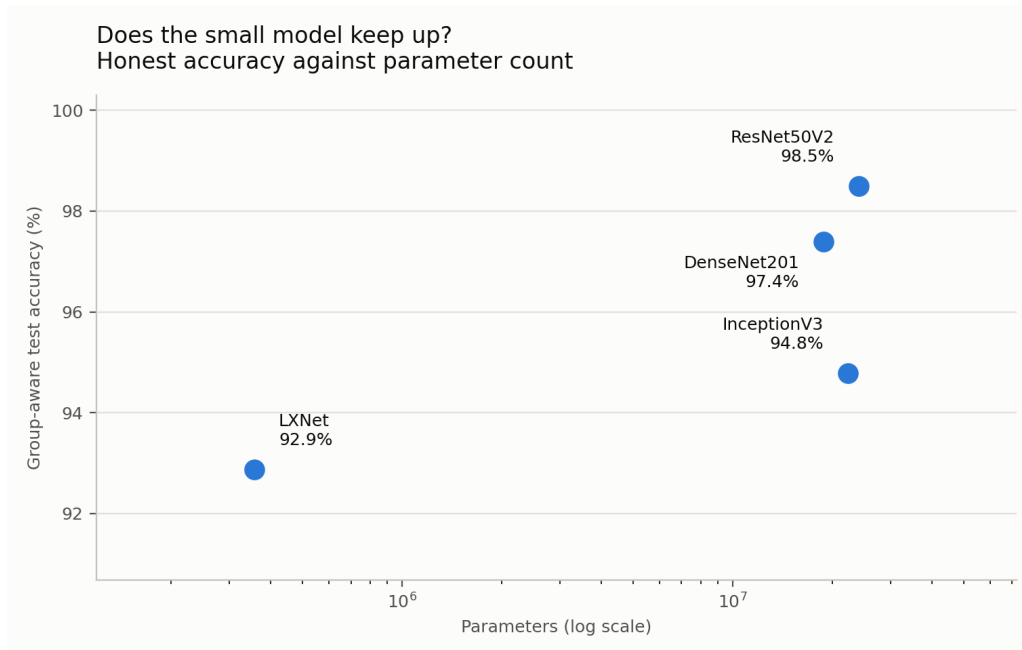


Figure 8.2: Group-aware test accuracy against parameter count (log scale). LXNET trails the largest backbone by 5.6 percentage points at $1/67^{\text{th}}$ the parameter count.

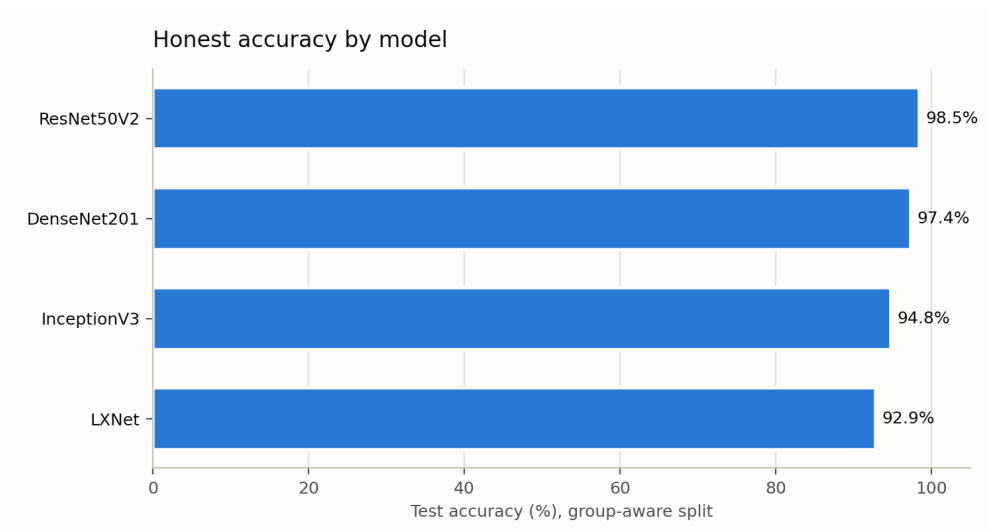


Figure 8.3: Honest (group-aware) accuracy by model, ranked.

Model	Params	Group-aware acc. (%)	Group-aware F1	Random-split acc. (%)	Inflation (pp)
ResNet50V2	24.09 M	98.5	0.988	—	—
DenseNet201	18.82 M	97.4	0.980	—	—
InceptionV3	22.33 M	94.8	0.953	—	—
LXNET	357 K	92.9	0.938	95.4	+2.5

Table 8.1: Hold-out comparison across both split protocols. Only LXNET was trained under the random-split arm (Section 8.8); the backbones were run group-aware only, for training-time reasons (Table 8.6).

Model	Accuracy	Precision (macro)	Recall (macro)	Macro-F1
ResNet50V2	98.50	98.70	99.02	0.9885
DenseNet201	97.39	97.77	98.20	0.9797
InceptionV3	94.78	95.13	95.69	0.9534
LXNET	92.88	93.16	95.06	0.9382

Table 8.2: Every headline metric, group-aware hold-out, all percentages except Macro-F1.

improvement in the efficiency metric (Table 8.6). Whether that trade is worth making depends entirely on the deployment target: a datacentre inference service has little reason to prefer LXNET over ResNet50V2, while a CPU-only edge device in a clinic without GPU access may not have the option to run the larger model at all.

8.4 Cross-Validation

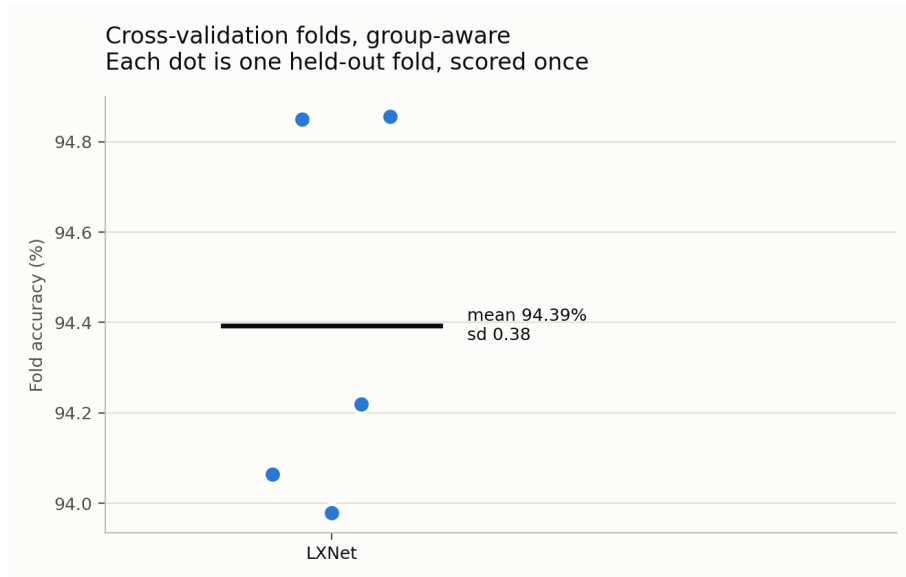


Figure 8.4: Per-fold accuracy, 5-fold group-aware cross-validation, LXNET only.

Each fold’s early-stopping monitor is carved out of that fold’s *training* rows (Section 6.3), so the reported fold is touched exactly once, at scoring time, and never influences which epoch’s weights are restored.

Model	Folds	Accuracy (%)	Macro-F1
LXNET (group-aware)	5	94.39 ± 0.38	0.9504
LXNET (random)	5	95.14 ± 1.55	—

Table 8.3: Cross-validation accuracy, both split protocols, identical model and preprocessing. Note the standard deviation: leakage does not only inflate the mean, it more than quadruples the fold-to-fold spread, because how much a random-split fold scores depends on how many of its images happen to have a twin sitting in that fold’s training portion.

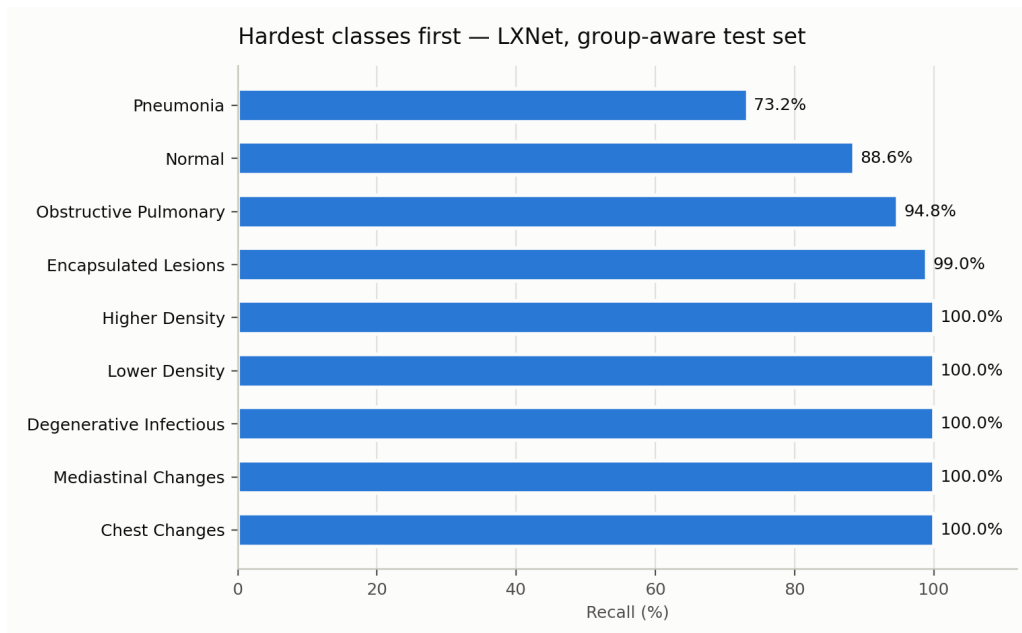


Figure 8.5: Per-class recall, LXNET, group-aware test set, hardest classes first.

8.5 Where the Errors Are

Seven of nine classes sit at 94–100% recall across every model; Pneumonia and Normal are the outliers, for every architecture tested, not just LXNET. This pattern — uniform difficulty concentrated on exactly two classes across four unrelated architectures — is treated in Chapter 13 as evidence that the easy classes are more likely separable by acquisition artefacts than by pathology, and is not resolved by this report.

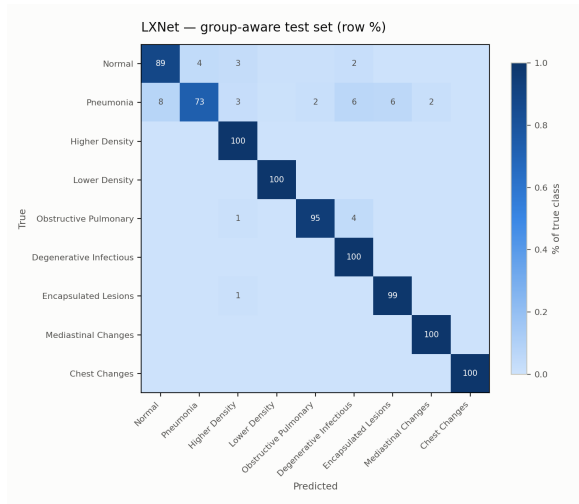
8.5.1 Confusion Matrices

8.6 Training Dynamics

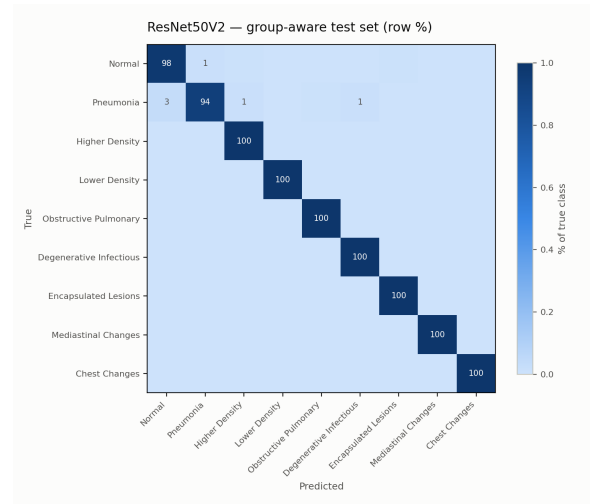
8.7 Training Cost

8.8 Significance Testing

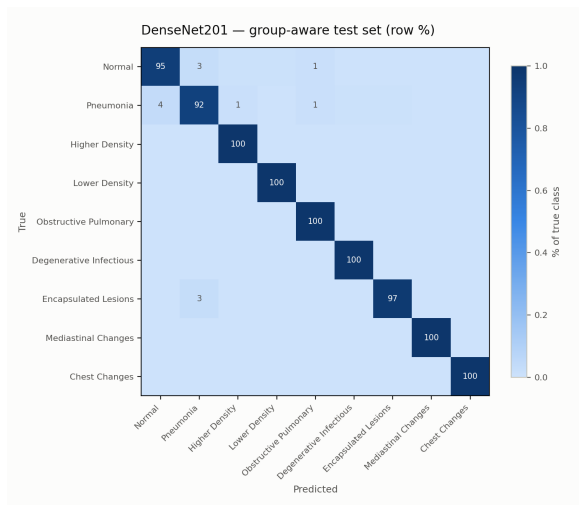
The Wilcoxon signed-rank comparison in Chapter 7 requires two models scored on the same set of cross-validation folds. In the measured run, only LXNET was cross-validated — the three backbones were run hold-out only, for the training-time reasons visible in Table 8.6 (five-fold cross-validation of ResNet50V2 alone would cost on the order of an hour of GPU time, repeated per model). This section is therefore reported as *not run* rather than populated with a single-



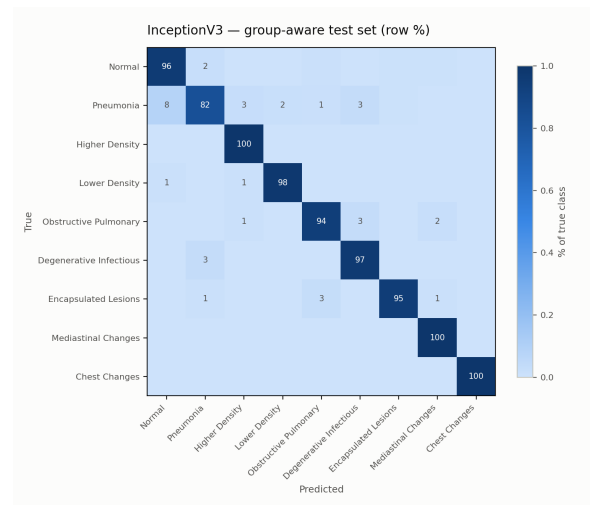
(a) LXNET



(b) ResNet50V2



(c) DenseNet201



(d) InceptionV3

Figure 8.6: Row-normalised confusion matrices, group-aware test set, all four models. Cell values are percentage of the true class's rows.

Class	Precision	Recall	F1	Support
Normal	93.7	88.6	0.910	201
Pneumonia	93.5	73.2	0.821	157
Higher Density	88.5	100.0	0.939	100
Lower Density	96.8	100.0	0.984	91
Obstructive Pulmonary	94.8	94.8	0.948	96
Degenerative Infectious	83.0	100.0	0.907	88
Encapsulated Lesions	91.4	99.0	0.950	97
Mediastinal Changes	96.7	100.0	0.983	88
Chest Changes	100.0	100.0	1.000	79

Table 8.4: Per-class detail, LXNET, group-aware test set.

Class	ResNet50V2	DenseNet201	InceptionV3	LXNet
Normal	97.5	94.5	96.0	88.6
Pneumonia	93.6	92.4	82.2	73.2
Higher Density	100.0	100.0	100.0	100.0
Lower Density	100.0	100.0	97.8	100.0
Obstructive Pulmonary	100.0	100.0	93.8	94.8
Degenerative Infectious	100.0	100.0	96.6	100.0
Encapsulated Lesions	100.0	96.9	94.8	99.0
Mediastinal Changes	100.0	100.0	100.0	100.0
Chest Changes	100.0	100.0	100.0	100.0

Table 8.5: Recall by class, every model, group-aware test set (%). The weakest cells — Pneumonia and Normal — are the two classes a screening tool exists to separate, and every model gives up most of its accuracy there.

arm comparison that would not mean anything. Cross-validating a second model is the direct way to populate it; Chapter 14 gives the exact command.

8.9 Dataset After Cleaning

- Byte-unique images retained: 6,657
- Byte-identical duplicates removed: 84
- Cross-class label-conflict groups removed: 1
- Split (train / val / test), group-aware arm: 4,660 / 1,000 / 997
- Split (train / val / test), random arm: 4,660 / 999 / 998
- Random-split test images with a training twin: 40.4%

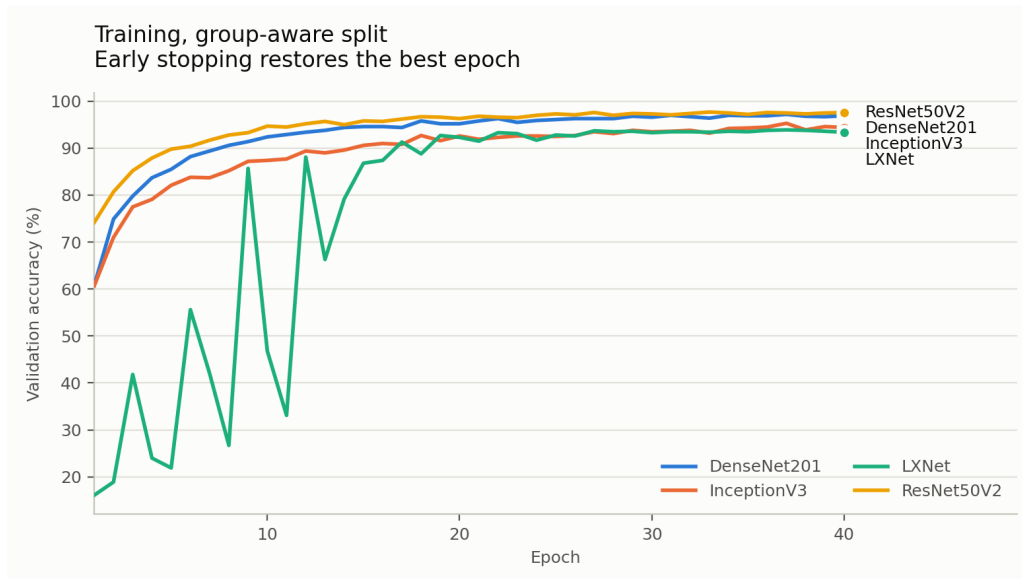


Figure 8.7: Validation accuracy per epoch, group-aware split. Early stopping restores the best-observed epoch per Table 6.1.

Model	Parameters	Epochs run	Wall clock	Accuracy per M params
LXNET	356,585	40	38m 01s	260.5
DenseNet201	18,816,073	40	27m 08s	5.2
InceptionV3	22,329,641	40	12m 19s	4.2
ResNet50V2	24,091,657	40	12m 54s	4.1

Table 8.6: Wall-clock training cost, single GPU, group-aware arm. The last column is deliberately crude (accuracy points per million parameters) but is the reference paper’s argument reduced to one number: LXNET extracts far more accuracy per parameter than any backbone tested, while still finishing below all of them in absolute terms. Note that LXNET trains *longer* in wall-clock time than two of the three frozen-backbone baselines despite having 50–70× fewer parameters — it is training every weight from scratch, while the baselines are only fitting a small head on already-frozen features.

Explainability

Three interpretability methods are implemented in `xai.py`, all attaching to the `final_conv` layer identified in Section 5.3. Each answers “which pixels drove this prediction” by a different mechanism, which is precisely why all three are shown side by side rather than just one: agreement across independent methods is weaker evidence of an artefact, and disagreement is itself informative.

9.1 Grad-CAM

Grad-CAM [2] weights each channel of the target convolutional layer’s feature map by the global-average-pooled gradient of the target class score with respect to that channel:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k} \quad (9.1)$$

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right) \quad (9.2)$$

where A^k is the k -th channel of the feature map, y^c is the score for class c before softmax, and the outer ReLU keeps only the evidence that positively supports the class, discarding channels whose gradient argues against it.

```

1 grad_model = tf.keras.Model(
2     model.inputs, [model.get_layer(layer_name).output, model.output]
3 )
4 with tf.GradientTape() as tape:
5     features, predictions = grad_model(batch)
6     if class_index is None:
7         class_index = int(tf.argmax(predictions[0]))
8     score = predictions[:, class_index]
9
10    gradients = tape.gradient(score, features)
11    weights = tf.reduce_mean(gradients, axis=(0, 1, 2))
12    heatmap = tf.reduce_sum(features[0] * weights, axis=-1).numpy()

```

Listing 9.1: Grad-CAM implementation (`lxnet/xai.py`).

9.2 Score-CAM

Score-CAM [3] removes the gradient entirely. Each channel of the feature map is upsampled to input resolution, used as a soft mask over the original image, and the *model’s own forward-pass*

score on that masked image becomes the channel’s weight:

$$L_{\text{Score-CAM}}^c = \text{ReLU}\left(\sum_k w_k^c A^k\right), \quad w_k^c = \text{softmax}(f(X \circ H_k)_c) \quad (9.3)$$

where H_k is channel k ’s upsampled, normalised activation map used as a mask, and $f(\cdot)_c$ is the model’s class- c score on the masked image. Evaluating every channel requires one forward pass per channel — hundreds for a 128-channel layer — so the implementation ranks channels by peak activation and keeps only the strongest `max_channels` (32 by default), since the weakest channels contribute negligibly to the final weighted sum and dominate the runtime disproportionately to their contribution.

9.3 LIME

LIME [4] is model-agnostic: it segments the image into superpixels, generates perturbed variants with random subsets of superpixels masked out, and fits a sparse linear model to the perturbed predictions to identify which superpixels are locally most responsible for the predicted class. `xai.lime_explanation` imports `lime` and `scikit-image` lazily, inside the function body, so the rest of the system — training, evaluation, inference — has no dependency on either package; they are declared as the `xai` optional extra in `pyproject.toml` and installed only when the interpretability panel is actually being rendered.

9.4 The Interpretability Panel

`explain.py` renders one row per class: the CLAHE’d input, then Grad-CAM, Score-CAM and LIME for the class the model actually predicted — correct and incorrect predictions both, and labelled as such. A panel built only from correct predictions would say nothing about whether the network is looking at lung tissue when it is wrong, which is the more diagnostically important question.

9.5 Reading the Maps Sceptically

Several attribution maps in Figure 9.1 are diffuse, and some place weight on image borders and edge artefacts rather than lung parenchyma. The panel is evidence about what the network’s final convolutional layer responds to; it is not a certificate that the network reasons clinically, and a plausible-looking heatmap centred on a lung is not proof of a correct mechanism — a network keying on a scanner-specific artefact that happens to correlate with lung position would produce an equally plausible-looking map. This distinction is treated at length in Chapter 13.

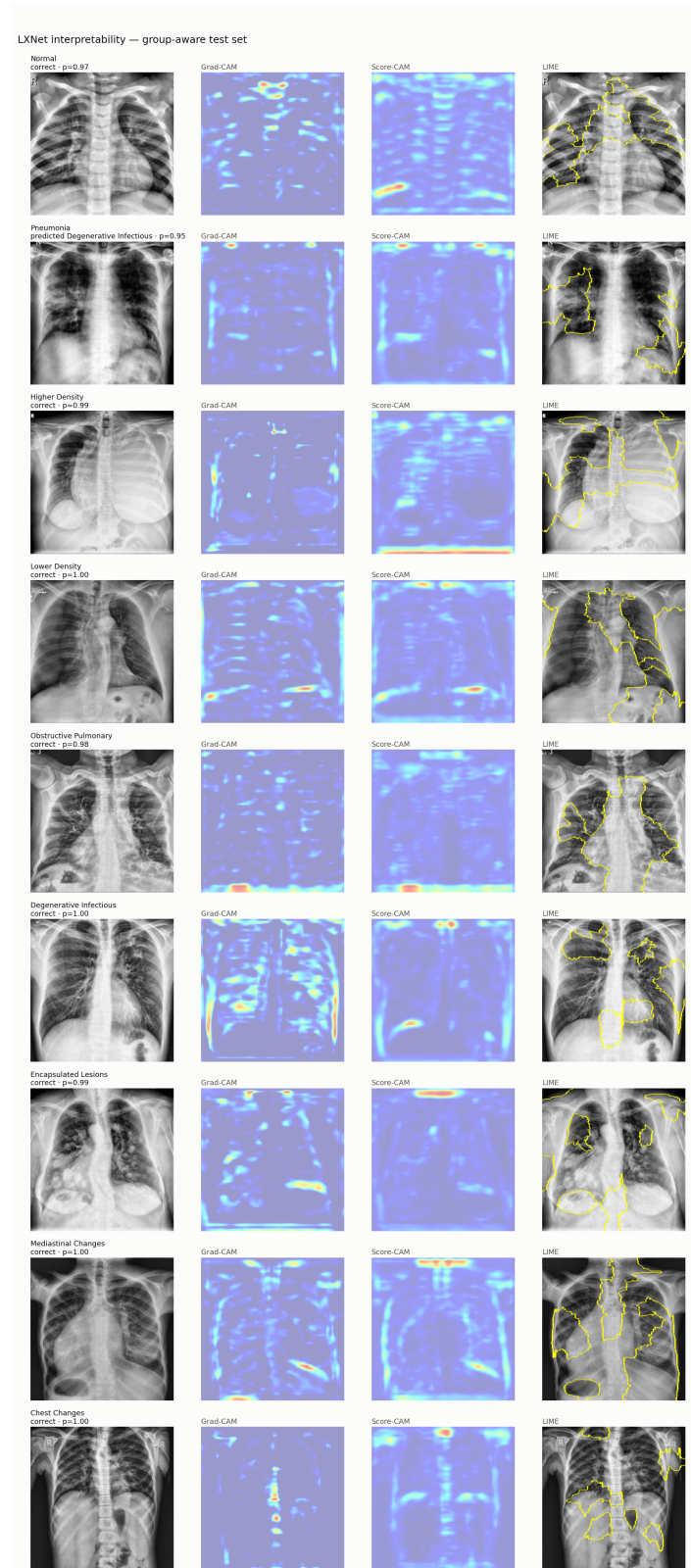


Figure 9.1: The full interpretability panel, one row per class, group-aware test set. Column 1: CLAHE'd input with the true class, correctness, and predicted-class probability. Columns 2–4: Grad-CAM, Score-CAM and LIME for the predicted class. Row 2 is a genuine failure — a Pneumonia film classified as Degenerative Infectious at $p = 0.95$ — shown because a panel of successes only would not demonstrate anything about failure behaviour.

CHAPTER 10

Inference and Deployment

10.1 The Trust Boundary

Every module upstream of `predict.py` operates on data this system has already indexed, hashed and audited. `predict.py` is where that assumption ends: its input is a file path handed in by a caller, about which nothing is known. `validate_image_path` checks existence, that the path is a file and not a directory, non-zero size, decodability (via `PIL.Image.verify`, which parses headers without decoding pixel data), and a minimum dimension of 32px — below which an image carries no diagnostic content once resized to 224×224 and is almost certainly a thumbnail, an icon, or a failed decode rather than a radiograph.

```
1 class InvalidImageError(ValueError):
2     """The input is not a usable image. Distinct from 'the model is
3     unsure'."""
4
5 def validate_image_path(path: str | Path) -> Path:
6     path = Path(path)
7     if not path.exists():
8         raise FileNotFoundError(f"no such file: {path}")
9     ...
10    if min(width, height) < MIN_DIMENSION:
11        raise InvalidImageError(
12            f"image is {width}x{height}; smaller than {MIN_DIMENSION}px "
13            f"carries no diagnostic content once resized to "
14            f"{IMAGE_SIZE[0]}x{IMAGE_SIZE[1]}")
15    )
16    return path
```

Listing 10.1: Validation happens before anything expensive touches the input (`lxnet/predict.py`).

`InvalidImageError` is a distinct exception type from `FileNotFoundError` deliberately: "this input cannot be classified" and "the model classified it with low confidence" are different failure modes, and a caller (or the CLI) needs to be able to tell them apart rather than catching a single generic exception and losing that distinction.

10.2 Preprocessing Parity

`predict.Classifier.predict` calls the identical `preprocess.load_image` function that `pipeline.build_cache` uses to build the training cache (Section 4.2), not a re-implementation of it. Preprocessing drifting between training and inference — a resize interpolation mode that differs, a CLAHE parameter that differs, a channel order that differs — is a classic production failure mode precisely because it does not raise an exception; it just quietly degrades accuracy

in a way that is easy to miss until someone notices the deployed model underperforms its evaluated numbers. `tests/test_predict.py` pins this by construction (Chapter 11).

10.3 The classifier Interface

`Classifier` loads a checkpoint once at construction (validating that the checkpoint file exists, with a message pointing at `lxnet-train` if it does not) and exposes `predict` (single image, full ranked distribution) and `predict_many` (batch over file paths, where one bad file does not abort the rest of the batch — each failure is caught, logged, and recorded per-path rather than propagated).

```

1 def predict_many(self, paths) -> dict[str, list[Prediction] | str]:
2     results: dict[str, list[Prediction] | str] = {}
3     for p in paths:
4         try:
5             results[str(p)] = self.predict(p)
6         except (FileNotFoundError, InvalidImageError) as exc:
7             log.warning("skipping %s: %s", p, exc)
8             results[str(p)] = f"error: {exc}"
9     return results

```

Listing 10.2: A batch of predictions is not aborted by one bad file (`lxnet/predict.py`).

Class name ordering is derived from `sorted(CLASS_LABELS)`, matching exactly how `data.index_dataset` assigns integer labels by sorted class-directory name (Chapter 3) — the code comment on this line is explicit that this must be sorted order, not dictionary insertion order, because the two coincide today only by construction of the `CLASS_LABELS` dictionary literal, not by any enforced invariant.

10.4 Command-Line Interface

`lxnet-predict` classifies one or more images, printing a ranked top- k (default 3) probability list per image, or machine-readable JSON with `--json`. A research-use disclaimer is printed to `stderr` by default (suppressible with `--quiet`) and is deliberately ASCII-only — LXNET’s default console encoding on Windows is `cp1252`, under which an em dash renders as a replacement character rather than the intended glyph. The process exit code is non-zero only when *every* input failed, so a caller can branch on total failure without treating one bad file in a batch as a fatal condition. Full flag reference: Table A.2.

10.5 Packaging

LXNET ships as a standard `setuptools`-backed Python package (`pyproject.toml`), installable with `pip install -e .` and exposing four console-script entry points (Tables A.1 to A.4). The TensorFlow dependency is pinned exactly at 2.10.1: it is the last TensorFlow release with native Windows GPU support, which is the platform this system was trained and evaluated on, and 2.11+ silently drops to CPU-only operation on Windows rather than failing to install — a pin, not a floor, and a direct consequence of the same GPU-visibility failure mode discussed in Section 6.4.

Testing and Continuous Integration

11.1 Design Philosophy

The system’s test suite is organised around a specific, narrow goal: pin the failure modes that produce a *plausible wrong number* rather than a crash. A stack trace is a cheap bug — it is loud, it blocks the run, and someone fixes it before results ship. The failure modes this project exists to prevent (Chapter 1) are the opposite: they produce a number that looks like a legitimate accuracy score and is quietly wrong. Every test file below is written against that threat model.

Test file	Guarantee pinned
<code>test_data.py</code>	Deduplication, cross-class conflict handling, and the ordering constraint between splitting and balancing (Section 3.5).
<code>test_grouping.py</code>	Perceptual near-duplicate detection and group-aware splitting — specifically, that no perceptual group is ever cut across a split boundary.
<code>test_preprocess.py</code>	CLAHE increases local contrast, preserves image geometry, and produces network-ready tensors from grayscale, RGB and RGBA sources alike.
<code>test_pipeline.py</code>	Image/label alignment survives the cache round-trip, and augmentation never reaches validation or test batches.
<code>test_models.py</code>	LXNET’s parameter count stays under the 500,000 budget (paper claim: ≈ 0.35 M); baseline construction produces the expected head shape.
<code>test_train_cli.py</code>	The GPU guard refuses CPU training by default (Section 6.4), and cross-validation’s early-stopping monitor is carved from training rows, not from the scored fold (Section 6.3).
<code>test_evaluate.py</code>	Macro averaging behaves correctly on imbalanced inputs; the Wilcoxon comparison handles the degenerate all-zero-difference case (Chapter 7).
<code>test_xai.py</code>	Grad-CAM and Score-CAM maps have the correct shape, are normalised, are class-dependent (a different requested class produces a different map), and are not constant — saliency bugs tend to produce a plausible-looking blob rather than an exception.
<code>test_explain.py</code>	The interpretability panel driver’s contract: correct row/column construction from a checkpoint and a set of examples.
<code>test_predict.py</code>	The inference trust boundary rejects invalid input before it reaches the model, and inference preprocessing matches training preprocessing bit-for-bit (Chapter 10).
<code>test_report.py</code>	<code>results.json</code> deterministically produces the expected tables and figure set.

Table 11.1: Test files mapped to the specific regression each one exists to catch, not just the module each one happens to cover.

11.2 Continuous Integration

Figure 11.1 outlines the two-job GitHub Actions workflow (`.github/workflows/ci.yml`). The dataset is not checked into the repository and the slow-marked tests download ImageNet weights, so CI runs exclusively against synthetic fixtures with `pytest -m "not slow"` and must stay green without either.

Job	Purpose
<code>test</code>	Lint (<code>ruff check</code>), format check (<code>ruff format --check</code>), and the full non-slow test suite with coverage.
<code>guarantees</code>	Re-runs specifically <code>test_grouping.py</code> , <code>test_data.py</code> , <code>test_train_cli.py</code> and <code>test_predict.py</code> as their own job, so a leakage or monitor-peeking regression fails as its own named check in the pull-request status list rather than being buried inside a hundred-line pytest summary.

Table 11.2: CI jobs, `.github/workflows/ci.yml`. Splitting `guarantees` out of `test` is a visibility decision, not a technical necessity — both jobs run the same suite on the same commit; the split exists so a reviewer scanning a checks list sees the evaluation-integrity guarantees fail by name.

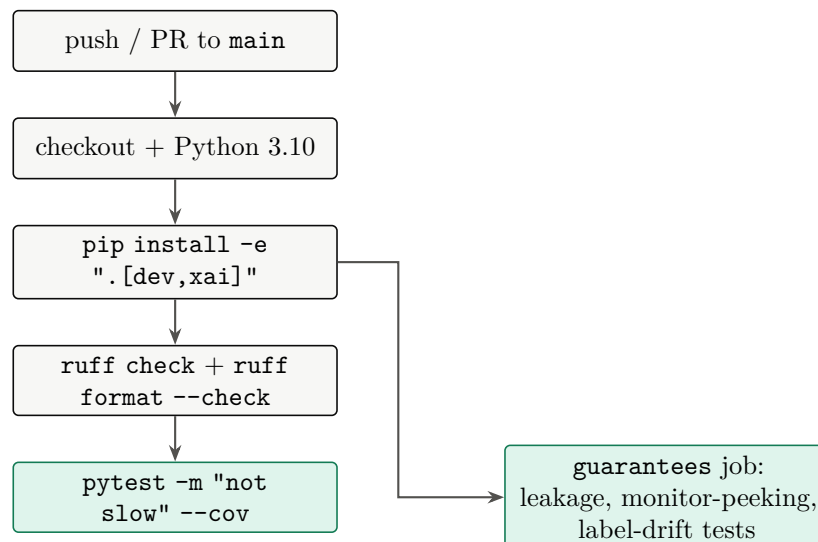


Figure 11.1: CI pipeline. `test` and `guarantees` run in parallel from independent checkouts; both must pass for the workflow to succeed.

CHAPTER 12

Engineering Decision Ledger

The preceding chapters justify each decision in place, next to the code it governs. This chapter collects them in one location, because a decision that is easy to find is a decision that is easy to revisit correctly rather than re-litigate from scratch.

Decision	Alternative considered	Why this way
Exact-match perceptual grouping (Section 3.3)	Fuzzy Hamming-distance matching	Measured: any tolerance ≥ 1 bit chains distinct images into a single 1,680-image group spanning 8 of 9 classes via transitive closure. Exact match is the only threshold that does not do this.
Split first, balance after (Section 3.5)	Balance the whole dataset, then split	The paper's ordering places duplicated minority-class images on both sides of the split, inflating recall on exactly the classes the technique was meant to help.
Carve the early-stopping monitor from each fold's training rows (Section 6.3)	Monitor <code>val_loss</code> on the fold being scored	<code>restore_best_weights</code> turns the reported score into a maximum over up to 40 epochs if the monitor and the scored fold are the same data.
Refuse to train on CPU by default (Section 6.4)	Fall back to CPU silently	TF 2.10 on Windows loses CUDA visibility silently when invoked outside an activated Conda environment, turning a 40-minute run into an overnight one with no error anywhere in the log.
Global average pooling, not Flatten (Chapter 5)	Flatten $\rightarrow$ Dense(256)	Flattening the final $28 \times 28 \times 128$ map costs 25.7 M weights on that layer alone — 70 $\times$ the entire network's actual budget.
CAM attachment on post-activation ReLU, not raw Conv2D (Section 5.3)	Attach to the raw convolution output	The raw output is pre-BatchNorm (arbitrary per-channel scale) and signed; Grad-CAM is defined over a non-negative, comparable feature map.
Cache preprocessing as grayscale <code>uint8</code> , not float32 RGB (Section 4.2)	Store the fully-prepared model-input tensor	12 $\times$ smaller on disk (338 MB vs. 4 GB); channel replication and float scaling are cheap enough to do inside the <code>tf.data</code> graph on every batch.

Decision	Alternative considered	Why this way
Macro-averaged metrics through-out (Chapter 7)	Micro (support-weighted) averaging	The dataset runs 1,340 Normal against 544 Chest Changes; micro averaging would let a model that neglects the smallest classes still post a flattering headline number.
Retain the leaky random-split protocol in the code-base (Chapter 2)	Delete it once the honest protocol was built	The leakage measurement in Chapter 8 is only meaningful because both protocols run on identical images, preprocessing and model code — deleting the leaky path would remove the ability to quantify what it cost.
Non-parametric paired significance test (Chapter 7)	A parametric paired <i>t</i> -test	Five cross-validation folds is far too few to justify assuming the per-fold score differences are normally distributed.
Read image bytes via buffer + <code>cv2.imdecode</code> , not <code>cv2.imread(path)</code> (Section 4.2)	<code>cv2.imread</code> directly on the file path	<code>cv2.imread</code> cannot open non-ASCII paths on Windows; the dataset's class directories are named in Portuguese.
Distinct <code>InvalidImageError</code> from <code>FileNotFoundError</code> (Chapter 10)	One generic exception type for all input failures	“This input cannot be classified” and “the file does not exist” are different failure modes a caller needs to branch on separately.

Table 12.1: Engineering decisions with an alternative that was measured or seriously considered and rejected, not merely available in principle.

CHAPTER 13

Limitations and Ethical Considerations

Not a Medical Device

This is not a medical device and must not be used to make clinical decisions. Every number in this report was measured on one public dataset of unknown clinical provenance, and none of it has been validated against a clinical reference standard.

13.1 Intended Use

13.1.1 In Scope

- Research on lightweight architectures for radiograph classification.
- A worked example of near-duplicate-aware evaluation, which is the purpose this repository exists to demonstrate.
- Education: a small model that trains end-to-end on one consumer GPU in approximately 40 minutes.
- A baseline to beat.

13.1.2 Explicitly Out of Scope

- Any clinical use: diagnosis, triage, screening, or second reads.
- Any use where a missed finding causes harm — Pneumonia recall is 73.2%.
- Populations, scanners, or acquisition protocols unlike the training corpus, which is itself undocumented — effectively, this excludes *any* population with confidence.
- Paediatric imaging, portable or bedside films, and lateral views. The training data's composition on these dimensions is unknown and unverified.

13.2 Numbered Limitations

- L1. Pneumonia recall is 73.2%.** This is the single most important number in this report. 8% of pneumonia films are classified *Normal* — a false negative on the most common actionable finding in the dataset. Any use that treats a *Normal* output as reassurance is a misuse of this system.

- L2. Normal recall is 88.6%.** The two classes a screening tool exists to separate — Normal and Pneumonia — are the two classes every model tested is worst at (Table 8.5). The remaining seven classes sit at 94–100% recall and carry the headline accuracy figure.
- L3. Measured accuracy is dataset-specific and probably optimistic.** Seven of nine classes scoring near-100% recall, uniformly across four architecturally unrelated models, is not a realistic difficulty profile for chest radiography as a diagnostic task. It more plausibly indicates that those classes are separable by acquisition artefacts — scanner make, exposure setting, framing, or collection source — than that any of the four models has learned the underlying pathology. This has not been tested directly and should be assumed true until disproven.
- L4. No external validation.** One dataset, one split protocol, and — probably, though provenance is undocumented — one institution’s worth of imaging. Nothing measured in this report predicts behaviour on a different hospital’s films, a different scanner vendor, or a different patient population.
- L5. Class definitions are coarse and non-standard.** “Higher Density” bundles effusion, consolidation, hydrothorax and empyema — clinically distinct entities with different causes and different management pathways. The nine-class label set used throughout this report is not a validated clinical taxonomy.
- L6. The interpretability maps are weak evidence, not proof.** Several Grad-CAM and Score-CAM maps in Figure 9.1 are diffuse or weight image borders rather than lung tissue. They demonstrate what the final convolutional layer responds to; they do not demonstrate clinical reasoning (Chapter 9).
- L7. No calibration.** Softmax outputs are not calibrated probabilities. A reported confidence of 0.95 is not a measured 95% chance of correctness — this system has not been assessed for expected calibration error, reliability diagrams, or behaviour under distribution shift.
- L8. No out-of-distribution detection.** Given a hand radiograph, an abdominal film, or an arbitrary photograph, the system returns a confident nine-class probability distribution over chest pathologies. It has no mechanism to recognise “this is not a chest X-ray.”

13.3 Training Data Provenance

Known and handled: 85 byte-identical duplicate files (removed before splitting); one image filed under two contradictory classes (both copies dropped, Section 3.2); 30% visual redundancy across the byte-unique corpus, addressed by group-aware splitting (Section 3.4); 40 distinct resolutions and mixed grayscale/RGB/RGBA colour modes, normalised in preprocessing (Chapter 4).

Unknown and unverifiable from the data as distributed: patient identities; whether a single patient contributes multiple films; scanner make, model, and acquisition parameters; geographic and demographic composition; label provenance, and whether labels were subject to any clinical adjudication. **No demographic breakdown of model performance is possible** on this dataset, and fairness across sex, age, or ethnicity is therefore untested — a serious and unresolved gap for any system that touches medical imaging.

13.4 Ethical Considerations

Deploying an automated reader into any diagnostic pathway shifts risk onto patients who did not consent to it and, in most workflows, cannot see it happening. The failure mode that matters clinically is not average accuracy but the missed finding: at 73% pneumonia recall, a system

trusted as a first-pass filter would send roughly one in four pneumonia patients away with a falsely reassuring result.

Automation bias compounds this risk independently of raw accuracy: a confident label presented to a time-pressured reader measurably shifts their judgement even when the label is wrong, and a plausible-looking interpretability overlay (Chapter 9) makes an incorrect prediction *more* persuasive to a human reviewer, not less.

Absent any demographic metadata, this system cannot be audited for differential performance across patient subgroups. A model that measures 93% accurate in aggregate can be substantially worse for an under-represented subgroup, and nothing in the current dataset or evaluation protocol would reveal that gap if it exists.

CHAPTER 14

Reproducibility

14.1 Environment

Component	Pinned version
Python	3.10 ($\geq 3.10, < 3.11$)
TensorFlow	2.10.1 exactly (Chapter 10)
NumPy	$\geq 1.23, < 2.0$
OpenCV (opencv-python)	≥ 4.7
Pillow	≥ 9.0
scikit-learn	≥ 1.2
SciPy	≥ 1.9
matplotlib	≥ 3.6
pandas	≥ 1.5
Optional (xai extra)	lime $\geq 0.2.0.1$, scikit-image ≥ 0.19

Table 14.1: Pinned dependency versions, `pyproject.toml`.

14.2 Reproducing the Reported Numbers

```
1 pip install -e ".[dev,xai]"
2
3 python -m lxnet.train --out-dir runs/grouped \
4     --split-mode grouped --cv-models LXNet
5
6 python -m lxnet.train --out-dir runs/random \
7     --split-mode random --models LXNet --cv-models LXNet
8
9 python -m lxnet.report
```

Listing 14.1: The exact commands that produced every table and figure in Chapter 8.

To populate the significance test left unrun in Section 8.8, cross-validate a second model against the same folds, for example:

```
1 python -m lxnet.train --out-dir runs/grouped \
2     --split-mode grouped --models ResNet50V2 --cv-models ResNet50V2
```

14.3 Reproducibility Record

Every figure in Chapter 8 is generated programmatically by `python -m lxnet.report` directly from `results.json`; none is drawn or edited by hand, and re-running the report step against

	Group-aware arm	Random arm
Split mode	grouped	random
Train / val / test	4,660 / 1,000 / 997	4,660 / 999 / 998
Seed	42	42
Max epochs	40, patience 8	40, patience 8
Batch size	32	32

Table 14.2: Reproducibility record. Both arms consume the same cached CLAHE’d tensor (Section 4.2), so preprocessing is bit-for-bit identical between them — the split function is the only variable that differs.

the same `results.json` is idempotent.

14.4 Hardware

Measured training times in Table 8.6 are single-GPU wall clock on the machine this replication was run on; absolute times will vary by GPU model and driver, but relative ordering (backbones finish in roughly a third the time of LXNET, despite 50–70× more parameters, because their weights are frozen) should hold generally, since it follows from the computation performed rather than from the specific hardware.

Conclusion and Future Work

15.1 Summary

This report documented a system built around one falsifiable measurement: that a naive evaluation split on this dataset overstates accuracy by 2.5 percentage points on the hold-out set and by a wider, less stable margin under cross-validation, because 30% of the corpus is redundant and a random split does not respect that redundancy. Removing the leak required changes at every layer — exact and perceptual deduplication, group-aware splitting and folding, balancing after rather than before splitting, and an early-stopping regime that cannot peek at the fold it reports — and each of those changes is now pinned by an automated test so it cannot silently regress (Chapter 11).

The result the system was built to test survives the correction, in a qualified form: LXNET does not match the largest backbone’s accuracy (92.9% versus ResNet50V2’s 98.5%), but it reaches that accuracy at 1/67th the parameter count, a trade that is legitimate engineering rather than an artefact of the evaluation protocol.

15.2 Future Work

- **Populate the significance test** (Section 8.8) by cross-validating at least one backbone alongside LXNET, so the parameter-efficiency claim can be stated with a p -value rather than a single hold-out comparison.
- **Test the acquisition-artefact hypothesis** raised in L3 (Chapter 13): if the seven near-100%-recall classes are separable by scanner or source metadata rather than pathology, a model trained on metadata alone should approach the same accuracy on those classes — a direct, falsifiable test of the concern rather than a standing caveat.
- **Carve the cross-validation monitor by perceptual group** (Section 6.3), closing the one documented piece of engineering debt in the training path, if the stopping epoch is ever observed to be systematically late.
- **Calibrate the output probabilities** (L7) and add out-of-distribution rejection (L8) before this system’s output could be responsibly surfaced to any downstream consumer, clinical or not.
- **External validation** on a second, independently-sourced chest radiograph dataset is the direct test of L4 and would be the single most informative next experiment this report does not contain.

APPENDIX A

Command-Line Interface Reference

A.1 `lxnet-train`

Flag	Type	Default	Meaning
<code>--data-root</code>	path	<code>dataset</code>	Dataset root directory.
<code>--out-dir</code>	path	<code>runs/latest</code>	Results, checkpoints, and preprocessing cache.
<code>--models</code>	list	all four	Subset of models to train.
<code>--epochs</code>	int	40	Maximum epochs (early stopping, patience 8).
<code>--batch-size</code>	int	32	Training batch size.
<code>--folds</code>	int	5	Cross-validation fold count.
<code>--seed</code>	int	42	Global random seed.
<code>--skip-cv</code>	flag	off	Hold-out evaluation only, no cross-validation.
<code>--allow-cpu</code>	flag	off	Permit training without a visible GPU (Section 6.4).
<code>--limit</code>	int	—	Subsample to N images, for smoke testing.
<code>--split-mode</code>	choice	<code>grouped</code>	<code>grouped</code> (leak-free) or <code>random</code> (leaky protocol, for comparison).
<code>--cv-models</code>	list	value of <code>--models</code>	Subset of models to cross-validate; the heavy backbones are usually hold-out only for time reasons.

Table A.1: `lxnet-train` / `python -m lxnet.train` flags.

A.2 `lxnet-predict`

A.3 `lxnet-report`

A.4 `lxnet-explain`

Flag	Type	Default	Meaning
<code>images</code>	path(s)	required	One or more image files to classify.
<code>--checkpoint</code>	path	<code>runs/grouped/checkpoints/LXNet_holdout.weights.h5</code>	Trained weights file.
<code>--model</code>	str	<code>LXNet</code>	Architecture matching the checkpoint.
<code>--top-k</code>	int	3	Classes to display (0 = all).
<code>--json</code>	flag	off	Machine-readable output.
<code>--quiet</code>	flag	off	Suppress the research-use disclaimer.

Table A.2: `lxnet-predict` flags.

Flag	Type	Default	Meaning
<code>--grouped</code>	path	<code>runs/grouped/results.json</code>	Group-aware arm results.
<code>--random</code>	path	<code>runs/random/results.json</code>	Random-split arm results (optional; leakage arm reads as pending if absent).
<code>--out-dir</code>	path	<code>docs</code>	Output directory for figures and <code>RESULTS.md</code> .
<code>--data-root</code>	path	<code>dataset</code>	Needed to recompute the leakage fraction reported in Chapter 8.

Table A.3: `lxnet-report` flags.

Flag	Type	Default	Meaning
<code>--run-dir</code>	path	<code>runs/grouped</code>	Directory holding the preprocessing cache and checkpoints.
<code>--model</code>	str	<code>LXNet</code>	Architecture to load and explain.
<code>--data-root</code>	path	<code>dataset</code>	Dataset root, to re-derive the test split.
<code>--out</code>	path	<code>docs/fig_xai.png</code>	Output panel image (Figure 9.1).
<code>--split-mode</code>	choice	<code>grouped</code>	Must match the checkpoint's training split mode.
<code>--seed</code>	int	42	Reproduces the same per-class example choice.
<code>--lime-samples</code>	int	400	LIME perturbation sample count (Chapter 9); higher is slower and more stable.

Table A.4: `lxnet-explain` flags.

APPENDIX B

Repository Layout

```
1 lxnet/  
2   data.py           indexing, dedup, perceptual grouping, splits, folds  
3   preprocess.py     CLAHE, image loading  
4   pipeline.py       cached tf.data input pipeline  
5   models.py         LXNet + DenseNet201 / ResNet50V2 / InceptionV3  
6   train.py          hold-out and k-fold orchestration  
7   predict.py        inference: validation, preprocessing, ranked  
                        probabilities  
8   evaluate.py       metrics, fold summaries, Wilcoxon signed-rank  
9   xai.py            Grad-CAM, Score-CAM, LIME, overlays  
10  explain.py        per-class interpretability panel  
11  report.py         themed figures and docs/RESULTS.md  
12 docs/              results, model card, figures, this report  
13 tests/             the evaluation guarantees are pinned here  
14 runs/              per-experiment checkpoints, caches and results.json  
15 .github/           CI workflow
```

References

- [1] Humayan, A. et al. *A Lightweight CNN for Lung Disease Classification from Chest X-ray with XAI-based Interpretability*. PLOS One, 2026.
- [2] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., Batra, D. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. IEEE International Conference on Computer Vision (ICCV), 2017.
- [3] Wang, H., Wang, Z., Du, M., Yang, F., Zhang, Z., Ding, S., Mardziel, P., Hu, X. *Score-CAM: Score-Weighted Visual Explanations for Convolutional Neural Networks*. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2020.
- [4] Ribeiro, M. T., Singh, S., Guestrin, C. “*Why Should I Trust You?*”: *Explaining the Predictions of Any Classifier*. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2016.
- [5] Pedregosa, F. et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2011.
- [6] Wilcoxon, F. *Individual Comparisons by Ranking Methods*. Biometrics Bulletin, 1(6), 1945.
- [7] fernando2rad. *X-Ray Lung Diseases Images (9 classes)*. Kaggle dataset. <https://www.kaggle.com/datasets/fernando2rad/x-ray-lung-diseases-images-9-classes>